BASES DE DATOS EN LA NUBE

Edición provisional para revisión. 26 de octubre de 2026.

Universidad Tecnológica de Xicotepec de Juárez. Ingeniería en Tecnologías de la Información e Innovación Digital. Optativa I: Bases de Datos en la Nube. Examen práctico de la Unidad 1.

Autor identificado: alan.cruz. Matrícula: 240516. Nombre completo: [COMPLETAR]. Grupo: [COMPLETAR]. Equipo: [COMPLETAR]. Docente: [COMPLETAR]. Fecha de evaluación: [CONFIRMAR]. Rama: Manual_VPS_BDs_Relacionales.

Se conserva el nombre FINAL solicitado; no certifica una entrega definitiva. Faltan requisitos obligatorios. Se explican procedimientos y resultados históricos sin ejecutar acciones en el VPS. Los originales y el PDF anterior se conservan.

Índice

1. Cómo utilizar este manual 3

2. Conceptos y requisitos previos 4

3. Preparación local en Windows 5

4. Acceso y revisión del VPS 10

5. Git, Docker Engine y Compose en el VPS 13

6. Comprender docker-compose.yml 15

7. Despliegue y descarga de imágenes 18

8. Estado, puertos, volúmenes y logs 22

9. Conexiones y pgAdmin 25

10. Bases, tablas y datos existentes 32

11. Usuarios, roles y privilegios 41

12. Auditoría y vistas de trazabilidad 44

13. Ocho apartados y diez casos oficiales 60

14. Respaldos completos de ambos motores 61

15. Errores y continuación de MySQL 63

16. Git y entrega 68

17. Conclusiones, limitaciones y referencias 69

Las páginas apaisadas contienen capturas completas, junto a los procedimientos que explican. Usa los marcadores para navegar.

1. Cómo utilizar este manual

1.1 Objetivo y fuentes

Aprenderás a distinguir Windows, VPS, servicios Docker y clientes SQL; revisar el despliegue propio de PostgreSQL, MySQL y pgAdmin; y reconocer qué resultados están demostrados. No se trata de reinstalar un entorno que ya funciona.

La fuente principal es la lista oficial de 56 indicadores y 100 puntos. La fotografía manuscrita aporta ocho apartados de simulación y pide respaldos de ambos motores. CUMPLIMIENTO_RUBRICA.md y su CSV contienen cada indicador literal. No se inventan casos para completar los diez solicitados.

1.2 Interpretar los estados

Histórico comprobado: una captura muestra un resultado en aquel momento, no el estado actual. Instrucción preparada: se explica qué hacer, sin ejecutarlo aquí. Pendiente: falta evidencia o una decisión. Bloqueado: depende del administrador. Una receta no equivale a una prueba aprobada.

Si ya tienes el proyecto desplegado, continúa por las lecturas y conserva los datos. Los pasos de instalación e inicialización corresponden solo a un entorno nuevo, propio y autorizado. No reimportes SQL completos sobre las bases existentes.

Se revisaron 35 capturas del documento de instalación y 22 de caps.docx: 57 originales diferentes. Esta edición incorpora 33, incluidas 17 de caps.docx. Las otras 24 se excluyen por secretos, información ajena, repetición o resultados insuficientes. No equivalen a 24 capturas nuevas pendientes. El inventario explica cada decisión.

2. Conceptos y requisitos previos

Un VPS es un servidor virtual accesible por Internet. Debian es su sistema operativo. SSH abre una terminal remota autenticada. Docker ejecuta procesos aislados, llamados contenedores; una imagen es su plantilla. Compose describe varios servicios. Un volumen conserva datos fuera de la vida del contenedor. Un puerto dirige conexiones a un servicio.

PostgreSQL y MySQL son motores distintos: cada uno tiene su propia bd_test_alan. Compartir nombre no comparte datos. pgAdmin administra PostgreSQL y Workbench administra MySQL. Git conserva archivos, pero no respalda automáticamente bases activas.

Dónde	Cómo reconocerlo	Qué se escribe
Windows / PowerShell	Terminal con PS	Git, clientes y acceso SSH
SSH del VPS	Sesión remota Debian	Consultas de herramientas y servicios propios
pgAdmin / Query Tool	Editor de conexión PostgreSQL	SQL PostgreSQL, no comandos Docker
Workbench / editor SQL	Pestaña conectada a MySQL	SQL MySQL, no PowerShell

Objetivo previo: reunir herramientas y acceso. Necesitas autorización, cuenta personal del sistema, dirección y huella SSH entregadas por canal seguro. SSH, pgAdmin y cada motor usan cuentas distintas.

1. Abre el repositorio en VS Code y localiza Practica_PostgreSQL sin mover las otras prácticas.

2. Ten disponibles PowerShell, navegador, Git y clientes compatibles con ambos motores.

3. Obtén los accesos privadamente; no escribas secretos en documentos ni capturas.

4. Si ya existe el despliegue, continúa por consultas. No reinstales para generar evidencia.

Verificación: puedes localizar Compose y los validadores sin abrir valores privados. Faltan recursos/cuentas/permisos del VPS: I01 e I02 en CAPTURAS_PENDIENTES.md.

3. Preparación local en Windows

3.1 Comprobar Git

Objetivo: identificar la herramienta de versiones. Dónde: PowerShell local, con Git instalado.

1. Abre Inicio, busca PowerShell y abre una ventana normal.

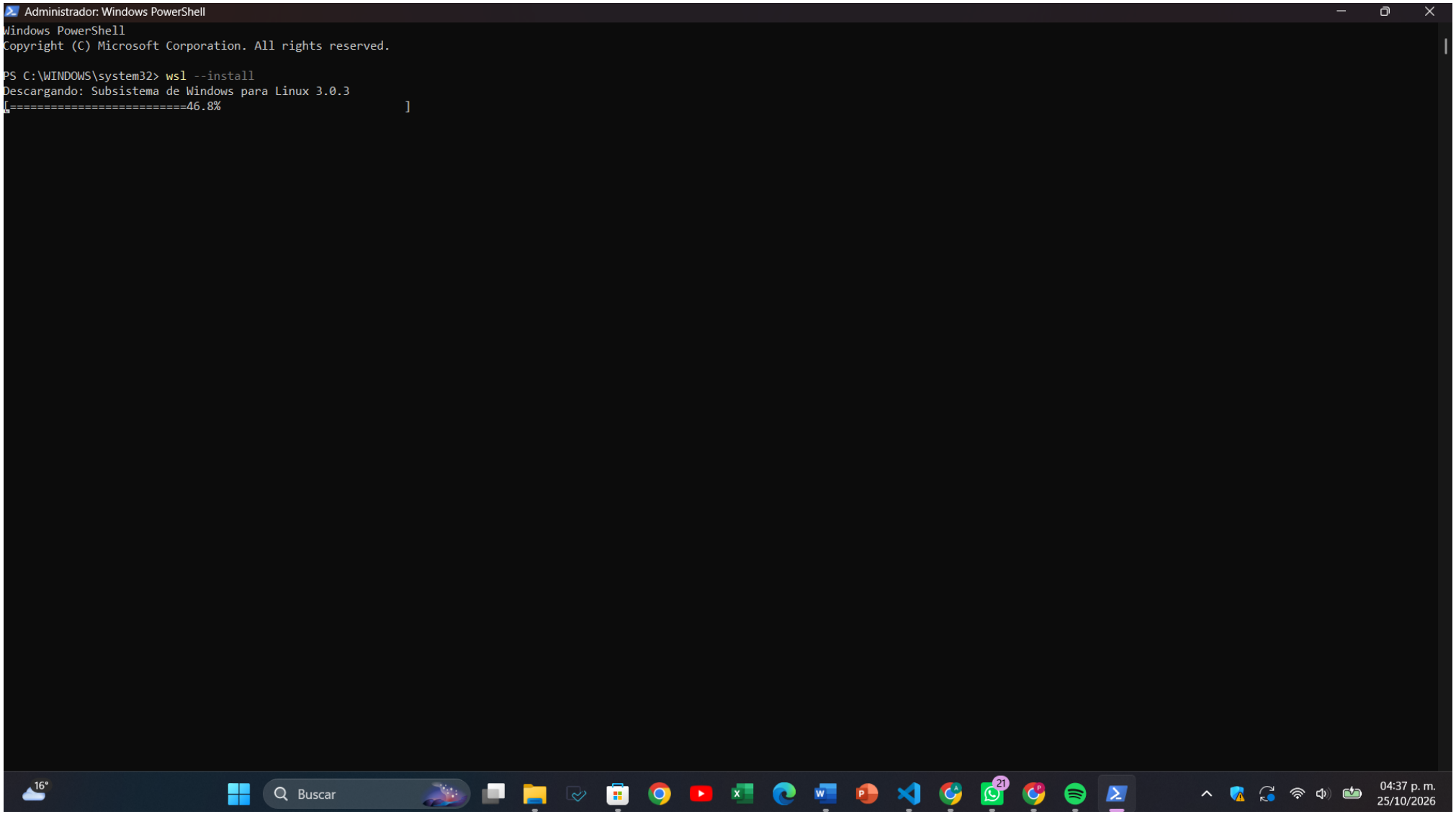
2. Escribe el comando y pulsa Enter.

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
git --version
```

Resultado esperado: git version y su número. Si no se reconoce, comprueba instalación y vuelve a abrir la terminal. No demuestra Git en Debian; véase capítulo 5.

Figura 1. DOCX-03



Git responde en Windows. Observa el número de versión; no es una prueba del VPS.

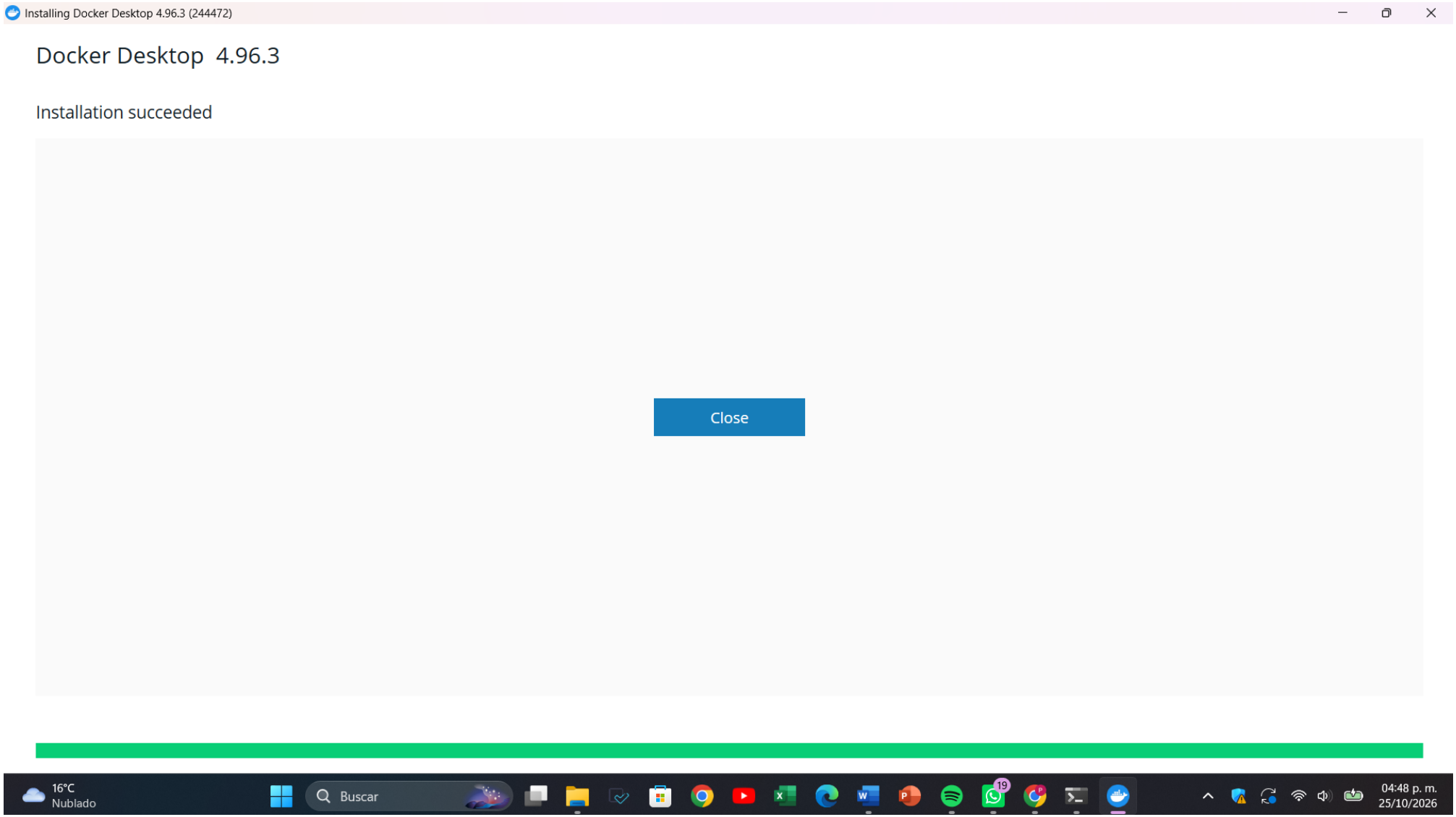
Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

3.2 Reconocer Docker Desktop

Docker Desktop local y Docker Engine del VPS son instalaciones distintas. Para un equipo nuevo autorizado, sigue el asistente oficial y los requisitos de Windows. Reconoce Installation succeeded, abre Docker Desktop y espera Engine running. Una lista vacía es normal antes de crear contenedores locales.

Verificación histórica: instalación terminada y motor local activo. No necesitas repetirla para continuar el VPS. Siguiente paso: distinguir la terminal remota.

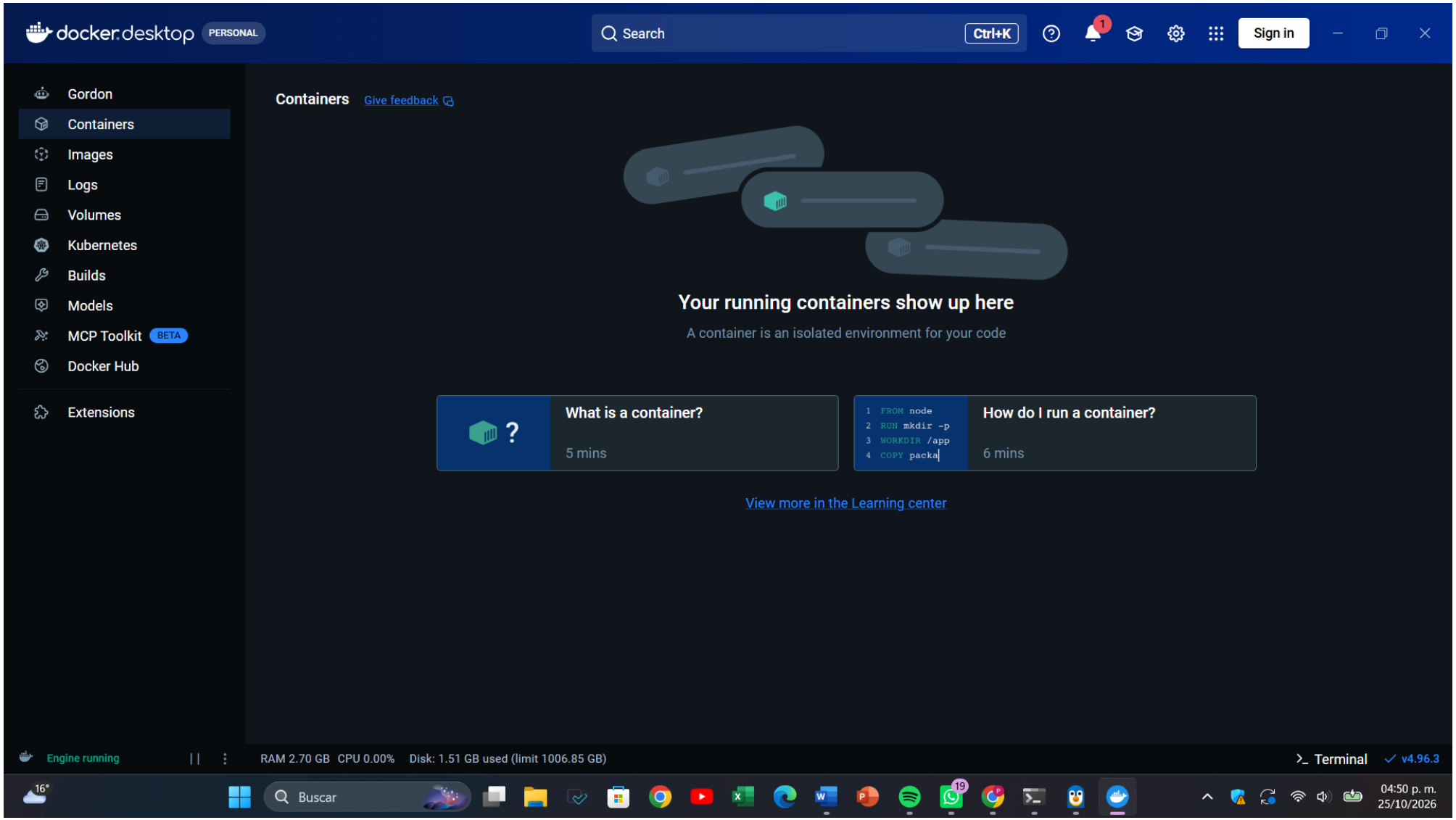
Figura 2. DOCX-04



El instalador local comunica Installation succeeded.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 3. DOCX-05



Docker Desktop con Engine running. La lista vacía no describe los contenedores remotos.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

4. Acceso y revisión del VPS

4.1 Abrir la conexión autorizada

Objetivo: entrar al servidor correcto. Dónde: PowerShell Windows. Requisitos: cuenta, dirección y huella confirmadas con el administrador. Sustituye los marcadores privadamente.

1. Abre una terminal local nueva; evita conectar desde otra sesión SSH por accidente.

2. Escribe:

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
ssh USUARIO_SO@HOST_VPS
```

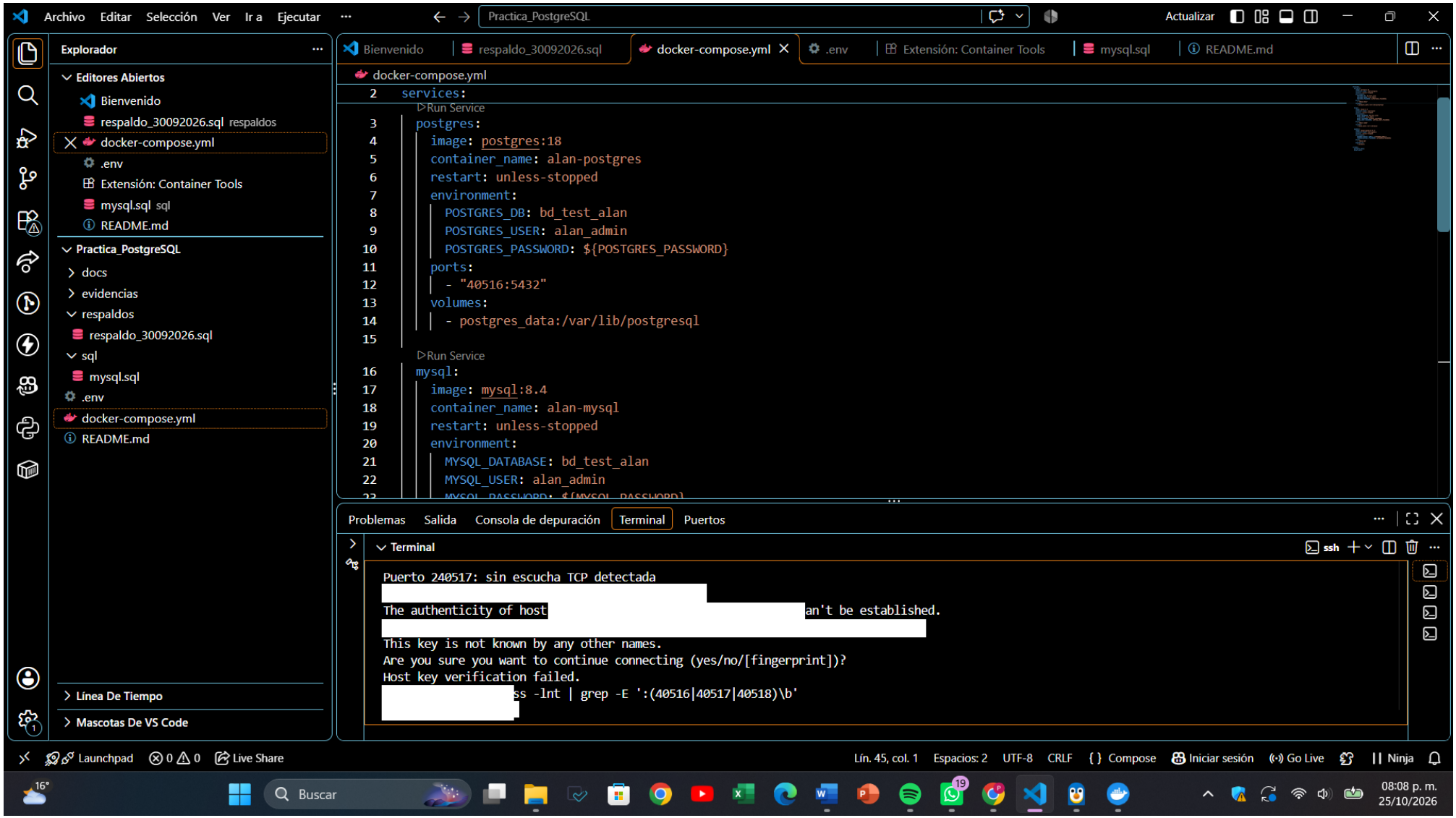
3. Compara la huella mostrada con la entregada por el administrador. No aceptes una desconocida para quitar el aviso.

4. Introduce la contraseña cuando se solicite; normalmente no se muestran caracteres.

5. Confirma que estás en Debian antes de escribir comandos del servidor.

Resultado esperado: terminal remota autenticada. La captura siguiente muestra un intento fallido dentro de otra sesión, no el primer acceso correcto desde Windows. Ante Host key verification failed, confirma host y huella; no borres claves conocidas ni desactives la verificación como atajo.

Figura 4. DOCX-14



Incidencia Host key verification failed, conservada sin datos privados de acceso.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

4.2 Consultar recursos

Dónde: SSH autenticado. Estos comandos leen, sin instalar ni reiniciar.

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
cat /etc/os-release
uname -m
nproc
free -h
df -h /
```

PRETTY_NAME identifica Debian; nproc muestra CPU, free -h memoria y df -h disco. Debian ya aparece en DOCX-11: faltan CPU/RAM/disco comprobables. Cuentas y permisos del equipo deben verificarse con el administrador, sin consultar hashes. Continúa con las herramientas del servidor.

4.3 Colocar el proyecto en un entorno nuevo autorizado

Objetivo: disponer del archivo correcto en el servidor. Este procedimiento no se ejecuta sobre el proyecto ya existente. Dónde: primero SSH para preparar un directorio propio nuevo; luego PowerShell local desde Practica_PostgreSQL. Requisito: permiso del administrador y comprobación de que no existe un despliegue previo que pueda sobrescribirse.

1. Si el proyecto ya está en el VPS, omite la transferencia y utiliza la copia existente revisada.

2. Solo para un entorno nuevo aprobado, crea el directorio propio en SSH. No cambies el de compañeros.

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
mkdir -p ~/Practica_PostgreSQL/sql
```

3. Desde PowerShell local, copia únicamente los archivos revisados al destino nuevo. No copies .env ni respaldos mediante un comodín.

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
scp ./docker-compose.yml USUARIO_SO@HOST_VPS:~/Practica_PostgreSQL/
scp ./sql/postgresql.sql USUARIO_SO@HOST_VPS:~/Practica_PostgreSQL/sql/
scp ./sql/mysql.sql USUARIO_SO@HOST_VPS:~/Practica_PostgreSQL/sql/
```

4. En SSH entra al directorio y lista los nombres, sin imprimir secretos.

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
cd ~/Practica_PostgreSQL
pwd
ls -l docker-compose.yml sql/
```

Resultado esperado: archivo Compose y SQL propios en el directorio autorizado. SCP puede sobrescribir archivos: no usarlo sobre un despliegue existente sin revisión. El administrador prepara el entorno privado por canal seguro. Falta captura de esta transferencia inicial; no se exige repetirla si la independencia del proyecto ya está demostrada por Compose y CAPS-01.

5. Git, Docker Engine y Compose en el VPS

Objetivo: comprobar antes de instalar. Dónde: SSH autorizado.

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
git --version
```

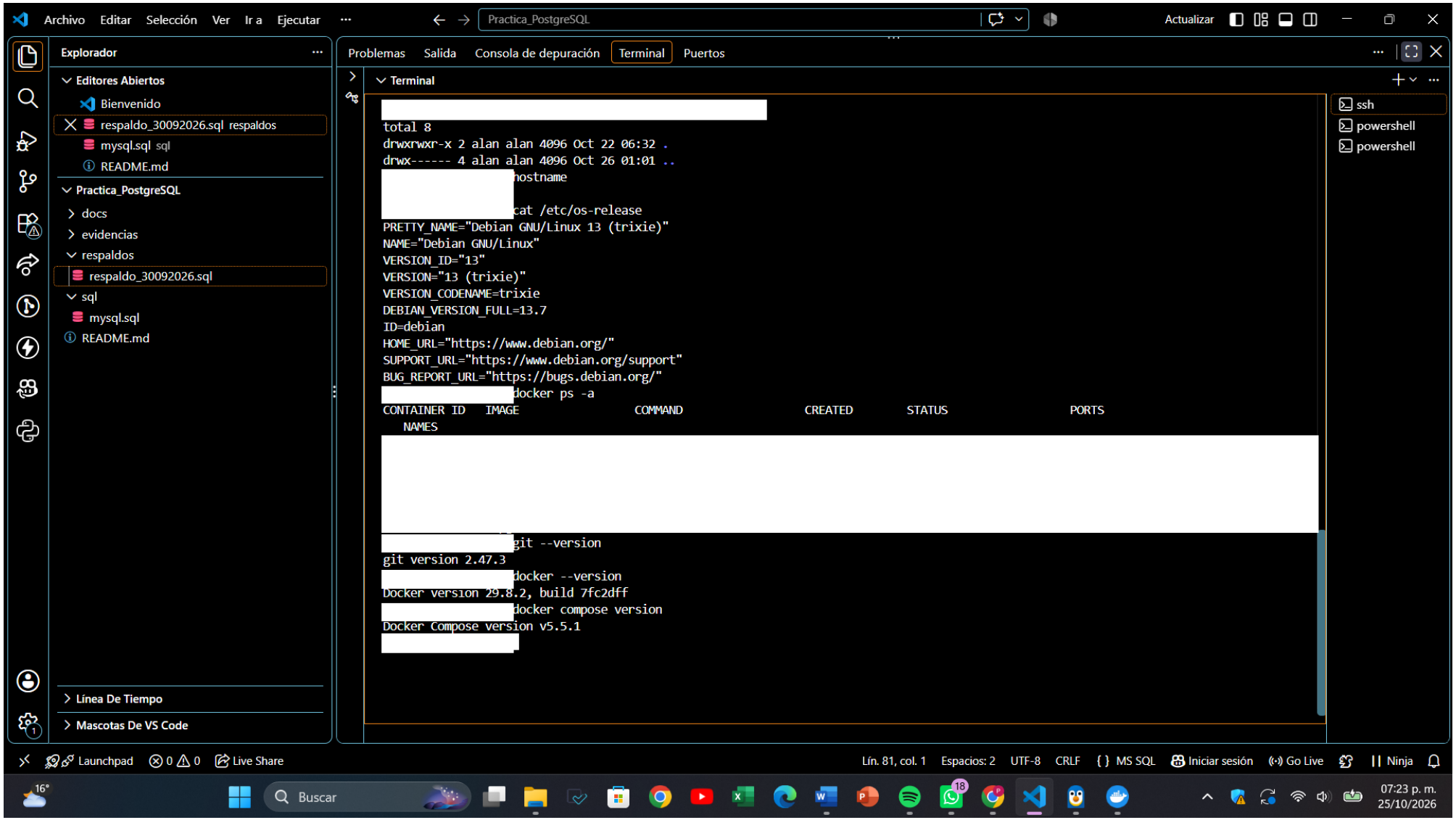
```
docker --version
```

```
docker compose version
```

Resultado esperado: tres versiones. La captura histórica muestra Git 2.47.3 y versiones Docker/Compose en Debian; no hace falta repetir esa evidencia. Ante Permission denied pide revisión del acceso; no cambies grupos ni uses privilegios por iniciativa propia.

Para un servidor realmente nuevo, el administrador sigue la instalación oficial de Docker Engine y Compose para Debian y comprueba Git mediante su gestor de paquetes autorizado: https://docs.docker.com/engine/install/debian/. No se reemplazan paquetes del VPS compartido existente. Un principiante debe completar esa preparación con el responsable antes de continuar. Verificación: funcionan los tres comandos. Siguiente paso: leer Compose.

Figura 5. DOCX-11



Debian y versiones de Git, Docker y Compose en el VPS. Se protegieron datos privados y contenedores ajenos.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

6. Comprender docker-compose.yml

6.1 Leer el archivo

Objetivo: saber qué se desplegaría. Dónde: VS Code, Practica_PostgreSQL. No abras valores de .env.

1. Selecciona docker-compose.yml en el explorador.

2. Localiza services y las entradas postgres, mysql y pgadmin.

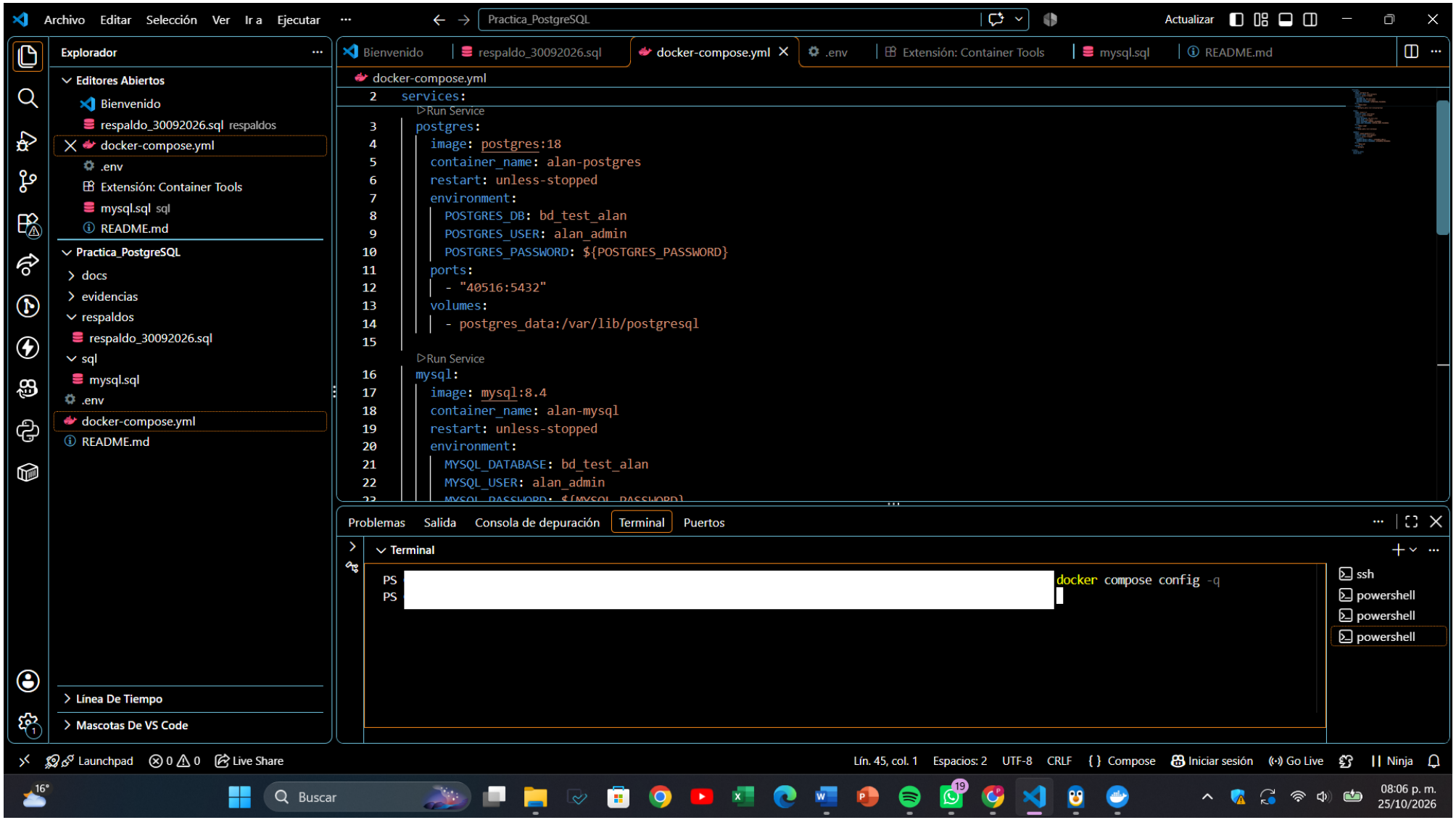
3. Identifica image como plantilla, container_name como nombre, environment como configuración, ports como externo:interno y volumes como almacenamiento.

4. Conserva nombres y puertos propios; no copies los de compañeros.

Servicio / contenedor	Imagen	Puerto externo : interno
postgres / alan-postgres	postgres:18	40516 : 5432
mysql / alan-mysql	mysql:8.4	40517 : 3306
pgadmin / alan-pgadmin	dpage/pgadmin4:9.17	40518 : 80

Windows utiliza el puerto externo. Dentro de Compose, pgAdmin conecta a postgres por 5432, no 40516. Ambos motores inicializan bd_test_alan con alan_admin; esto no prueba sus privilegios efectivos.

Figura 6. DOCX-13



Archivo Compose y validación silenciosa: imágenes, puertos y referencias a variables sin valores privados.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

6.2 Variables y persistencia

Las cinco variables son POSTGRES_PASSWORD, MYSQL_PASSWORD, MYSQL_ROOT_PASSWORD, PGADMIN_EMAIL y PGADMIN_PASSWORD. .env.example contiene ejemplos ficticios; el archivo privado se prepara sin capturarlo. No publiques configuración resuelta.

PostgreSQL monta postgres_data en /var/lib/postgresql, y MySQL mysql_data en /var/lib/mysql. Se conserva el montaje de PostgreSQL 18. pgAdmin no declara volumen propio; su persistencia no está demostrada. No se añade uno encima de datos sin estudiar sus efectos. depends_on ordena dependencias, pero no garantiza que el motor acepte consultas.

6.3 Validar sin desplegar

Dónde: PowerShell en Practica_PostgreSQL. Usa únicamente la plantilla pública:

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
docker compose --env-file .env.example config --quiet
```

Resultado esperado: termina sin errores y sin salida. Si falla, revisa sangría y variables. Valida el archivo local, no el VPS. Continúa con el despliegue y su revisión.

7. Despliegue y descarga de imágenes

7.1 Inicialización nueva autorizada

Objetivo: iniciar servicios propios. Dónde: SSH en tu directorio del proyecto. Requisitos: aprobación del administrador, puertos libres, variables privadas y entorno nuevo sin datos en riesgo. Esta explicación no se volvió a ejecutar sobre el despliegue existente.

1. Confirma directorio y Compose propios.

2. El administrador revisa los puertos 40516, 40517 y 40518 y el acceso de red.

3. Valida con config --quiet sin imprimir secretos.

4. Solo para la primera puesta en marcha aprobada:

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
docker compose up -d
```

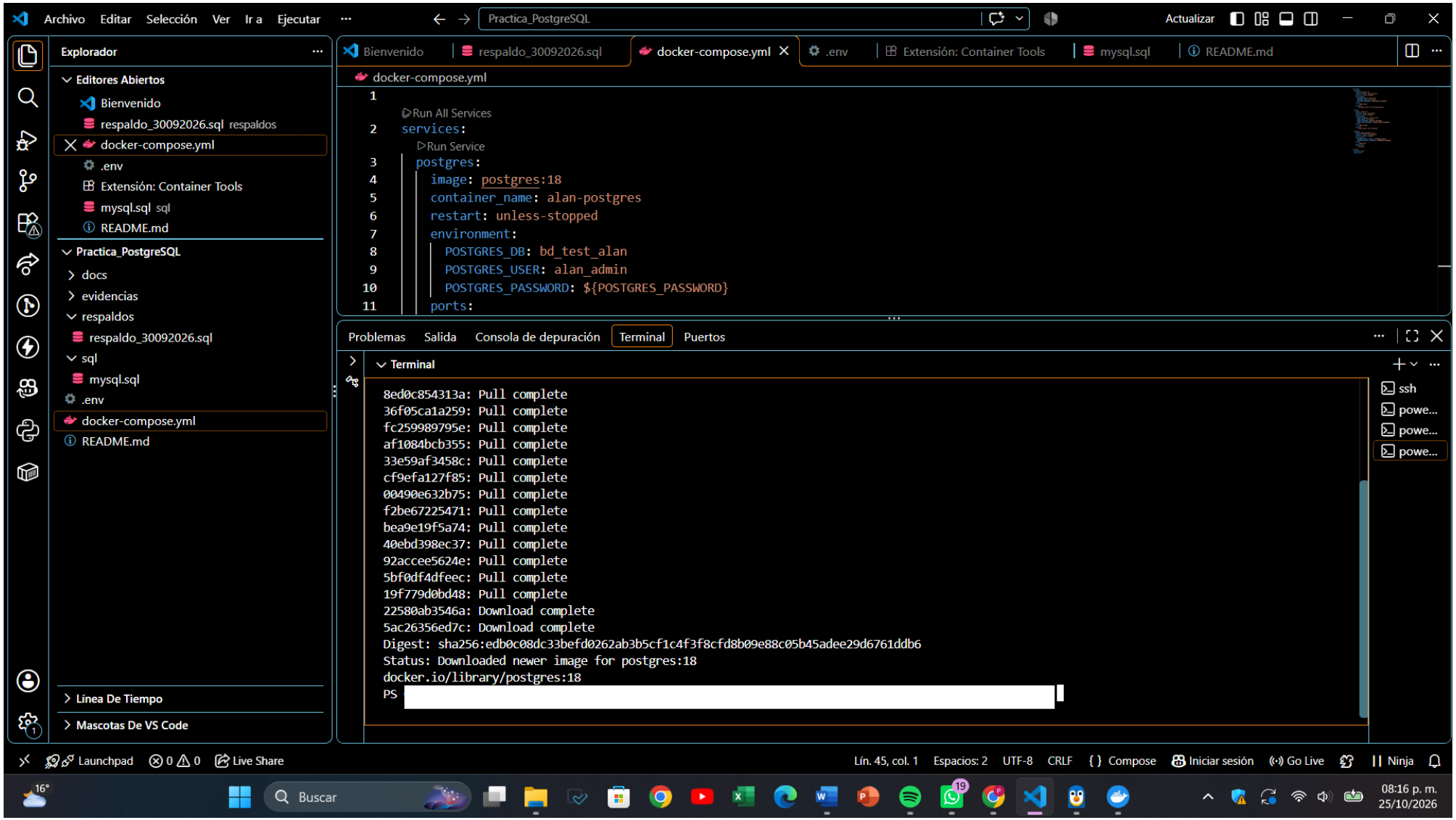
up puede crear o recrear contenedores; -d los deja en segundo plano. No se usa como comprobación inocua de servicios activos. Los mensajes de descarga describen imágenes, no tablas ni respaldos.

7.2 Reconocer descargas

Las siguientes imágenes son de Windows, no de un despliegue nuevo del VPS. Cada equipo mantiene sus propias imágenes. Reconoce Pull complete y Downloaded newer image; no repitas docker pull si ya existen.

Resultado esperado de una inicialización nueva: servicios propios activos. La evidencia del VPS aparece a continuación; no reinicies ni borres volúmenes para repetirla.

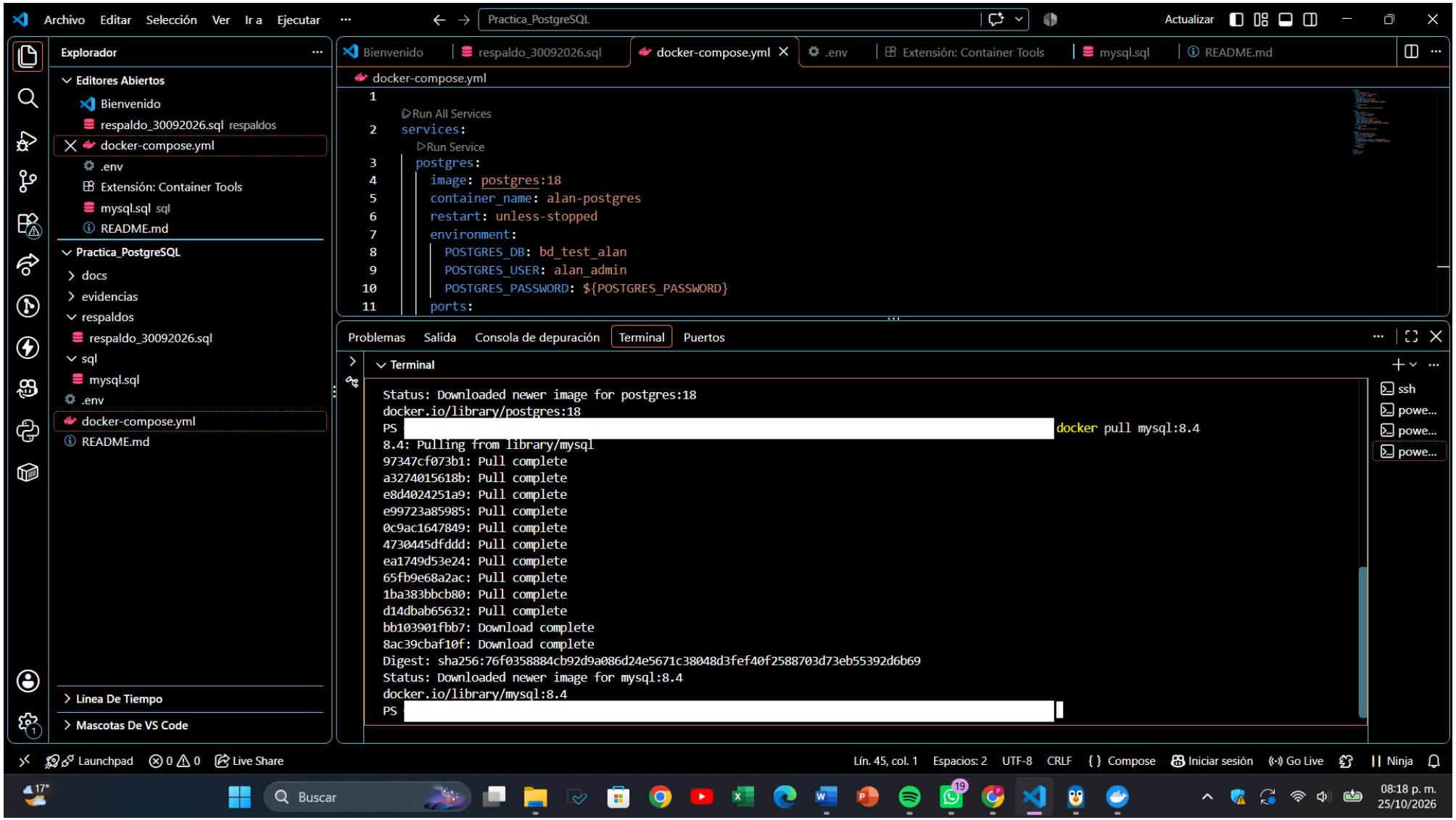
Figura 7. DOCX-18



Descarga local de postgres:18 completada. Las capas no son datos de la base del VPS.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

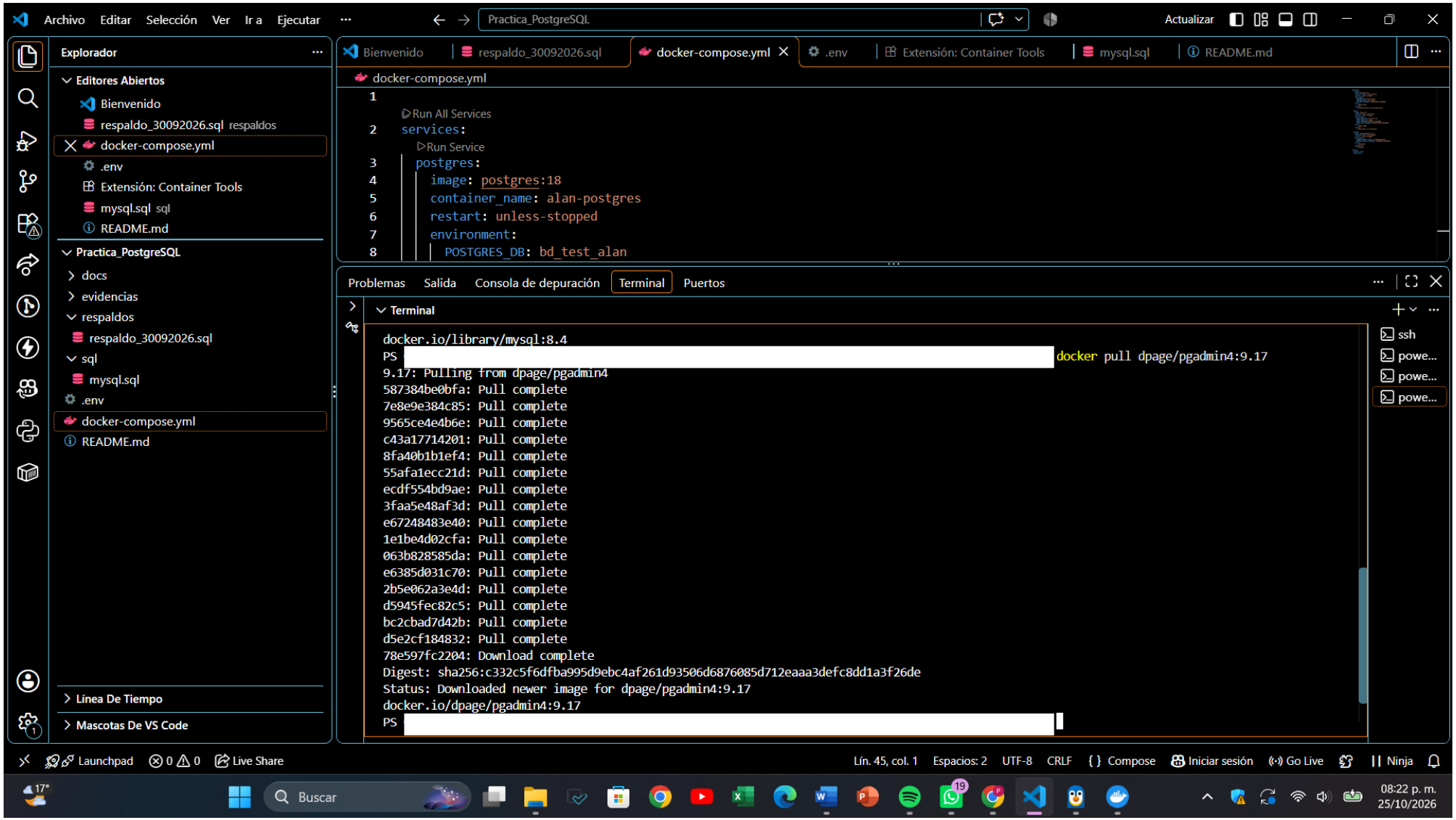
Figura 8. DOCX-19



Descarga local de mysql:8.4. Descargar no equivale a conectar al motor.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 9. DOCX-20



Descarga local de pgAdmin 9.17. Todavía faltan sesión y registro del servidor.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

8. Estado, puertos, volúmenes y logs

8.1 Revisar servicios propios

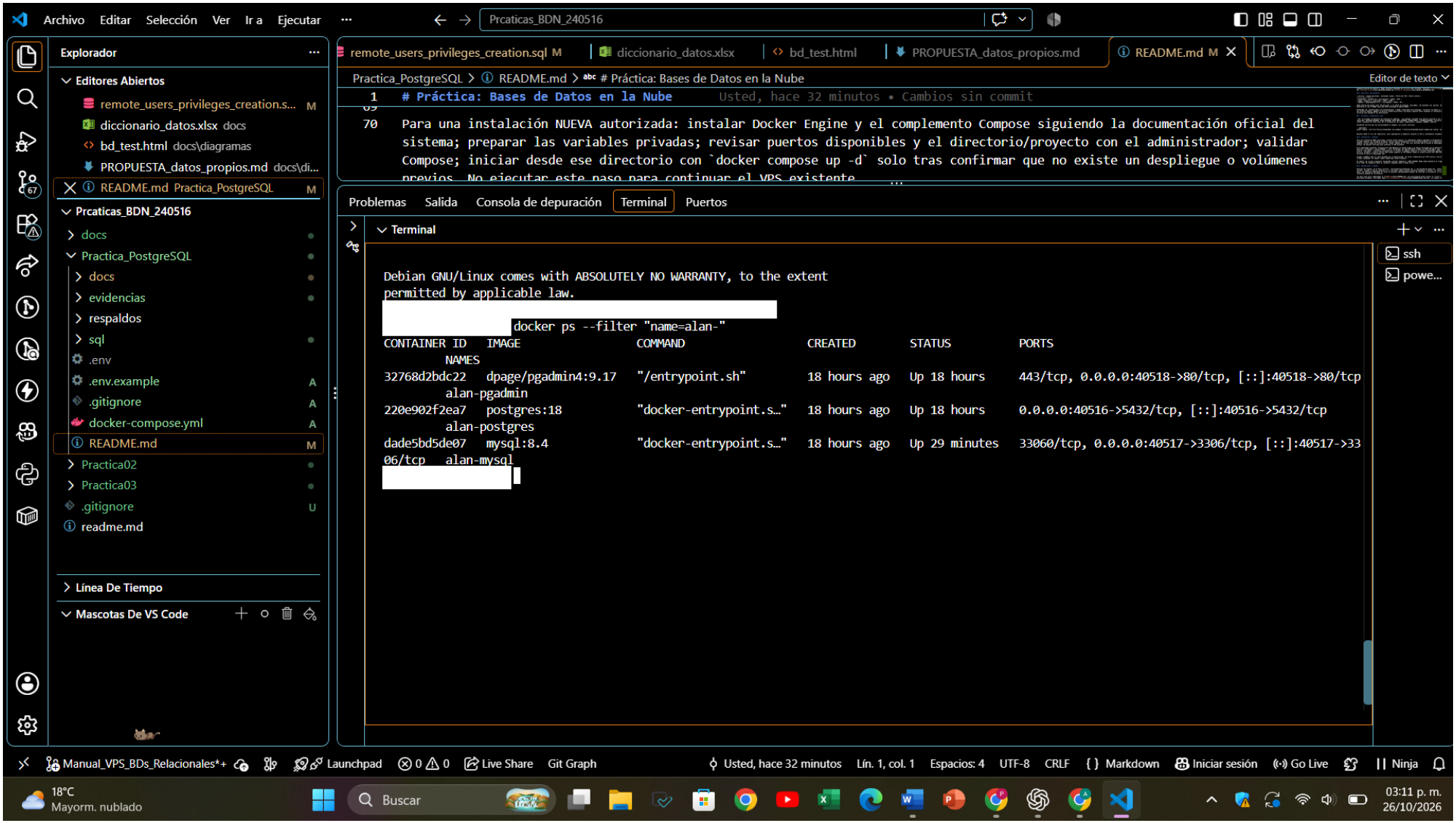
Objetivo: consultar el estado sin cambiarlo. Dónde: SSH autorizado.

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
docker ps --filter name=alan-
```

Verifica alan-postgres, alan-mysql y alan-pgadmin. STATUS Up indica proceso activo, no salud SQL ni ausencia de errores. PORTS muestra el mapeo. No uses contenedores ajenos como evidencia propia.

Figura 10. CAPS-01



Tres servicios propios y puertos publicados. Up no sustituye consultas SQL o revisión de logs.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

8.2 Consultar montajes y registros

Lecturas preparadas, no ejecutadas aquí. Los nombres explícitos evitan inspeccionar otros servicios.

Comando / consulta (bash). Revisar el lugar indicado antes de ejecutar.

```
docker inspect alan-postgres --format '{{json .Mounts}}'
```

```
docker inspect alan-mysql --format '{{json .Mounts}}'
```

```
docker logs --tail 50 alan-postgres
```

```
docker logs --tail 50 alan-mysql
```

```
docker logs --tail 50 alan-pgadmin
```

Resultado esperado: montajes coherentes con Compose y mensajes sin fallos pendientes. Revisa privacidad antes de capturar logs. No publiques docker inspect completo: puede mostrar variables privadas. Persistencia efectiva y logs completos requieren evidencia; C01 detalla el alcance sin ordenar recrear servicios.

8.3 Verificar red

Publicar puertos no demuestra reglas correctas del firewall. Solicita al administrador evidencia autorizada de acceso y ausencia de conflictos, I03. No abras reglas por tu cuenta. Ante timeout revisa host, puerto, servicio y red; no cambies contraseñas a ciegas. Continúa con clientes externos.

9. Conexiones y pgAdmin

9.1 Abrir la interfaz web

Objetivo: entrar a pgAdmin distinguiendo su cuenta de la cuenta PostgreSQL. Dónde: navegador Windows. Requisitos: dirección y acceso privados aprobados.

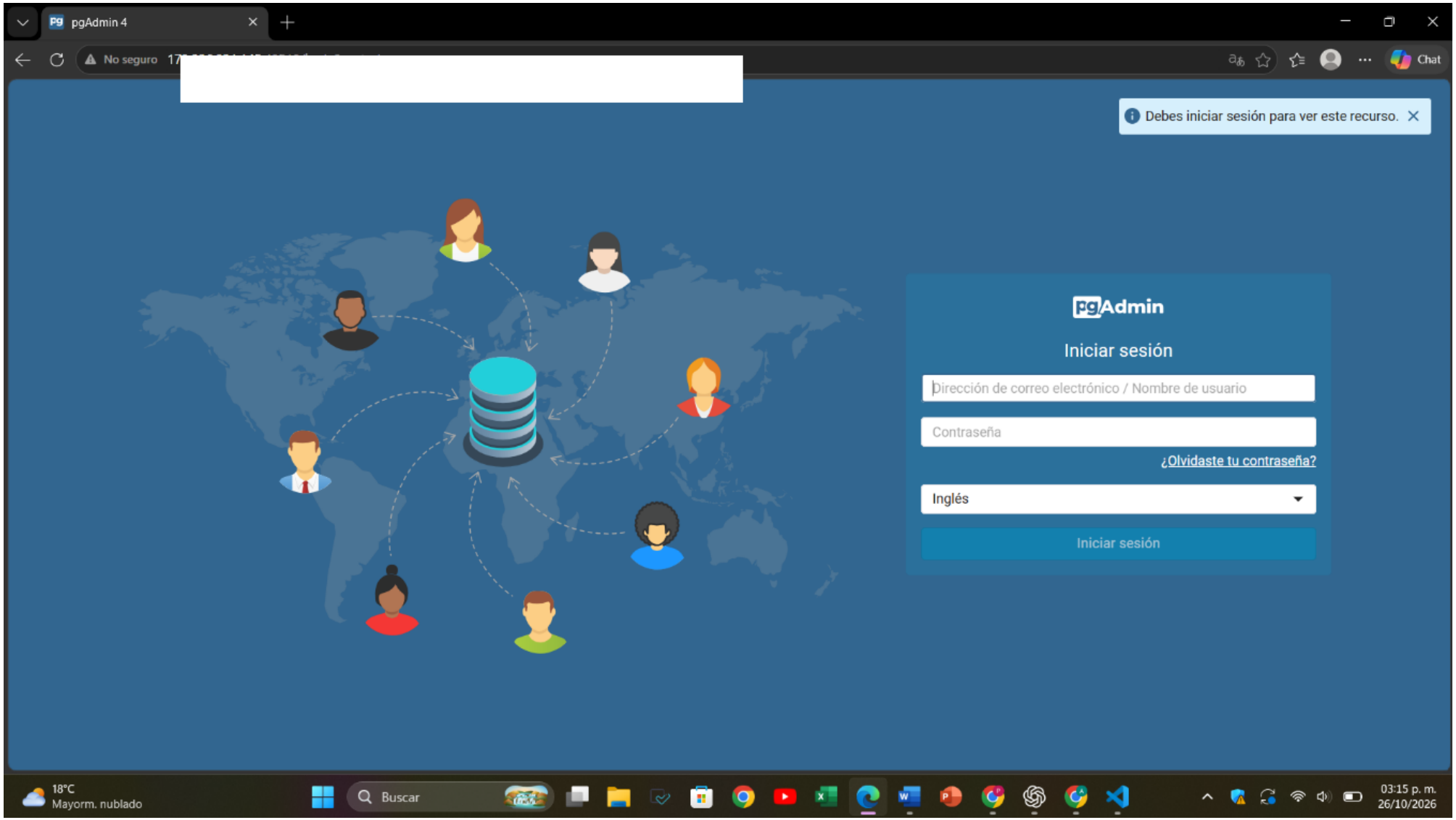
1. Abre la dirección autorizada para 40518, sin publicarla en el reporte.

2. Introduce los datos privados en el formulario sin capturarlos.

3. Pulsa Iniciar sesión y comprueba el escritorio.

La evidencia histórica muestra HTTP con aviso No seguro; no acredita TLS. Solicita el canal seguro aprobado antes de transmitir credenciales. No se configura aquí proxy o túnel. El formulario no demuestra autenticación: la siguiente pantalla sí muestra una sesión abierta.

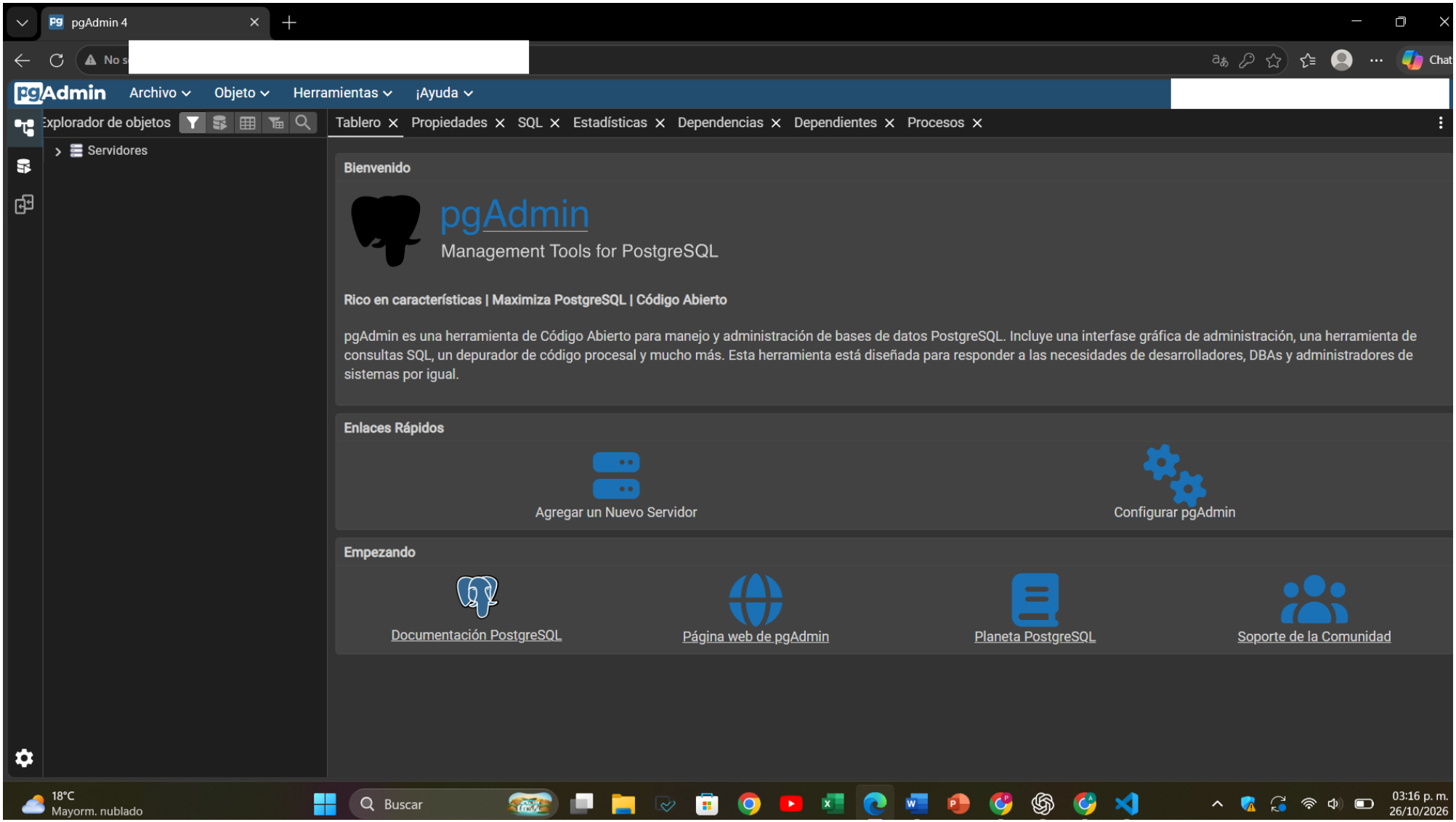
Figura 11. CAPS-04



Formulario pgAdmin sin datos de entrada. Dirección privada eliminada de la copia.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 12. CAPS-05



Escritorio de pgAdmin tras iniciar sesión. Dirección e identificador de acceso protegidos.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

9.2 Registrar PostgreSQL

Objetivo: asociar el cliente con el motor correcto. Requisitos: sesión pgAdmin, base y cuenta autorizadas. La conexión al motor sale del servicio pgAdmin dentro de Compose, aunque controles su web desde Windows.

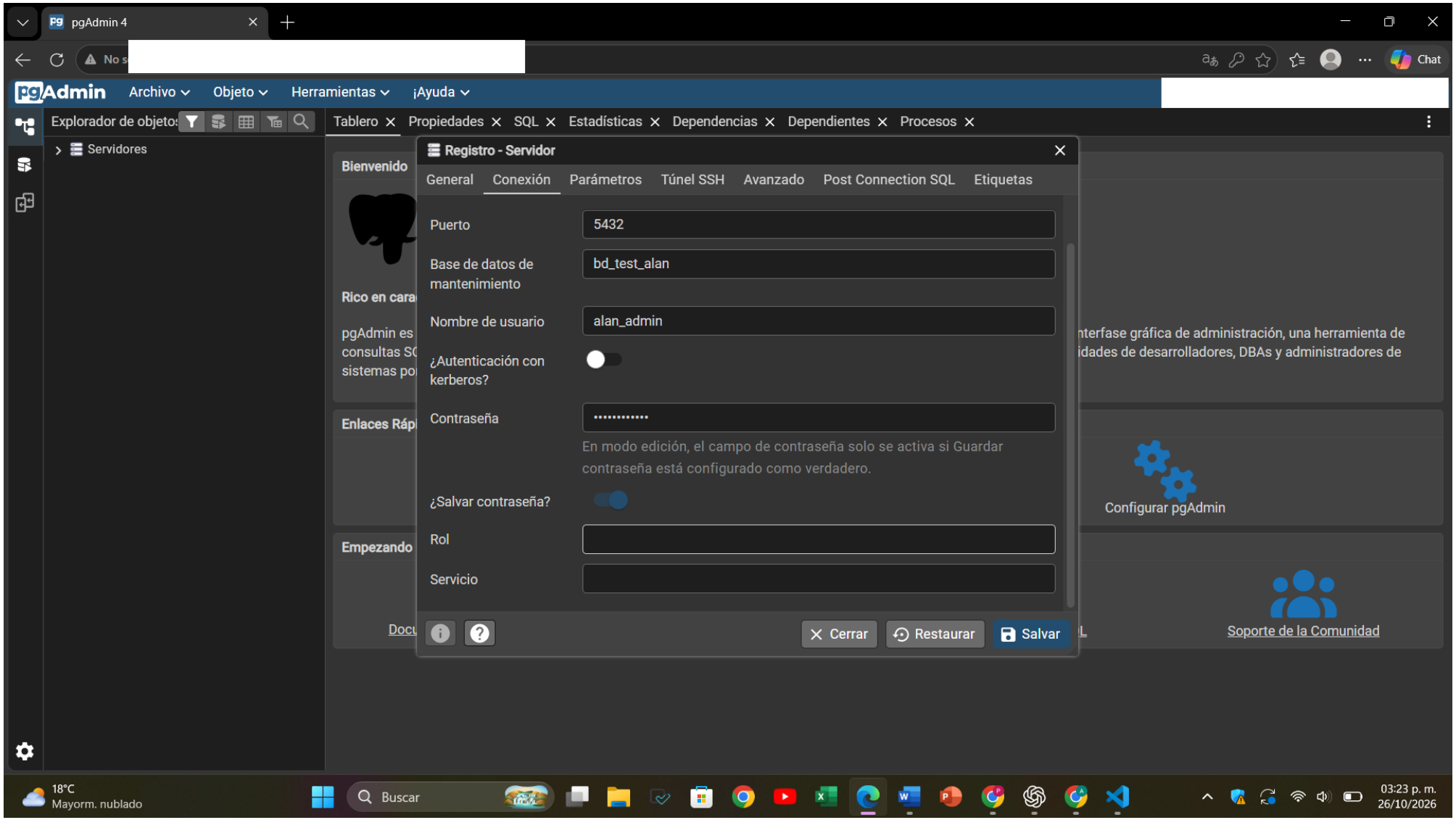
1. Botón derecho en Servers; elige Register > Server.

2. En General escribe una etiqueta propia, por ejemplo PostgreSQL - Alan; no renombra la base.

3. En Connection indica host postgres, puerto 5432, Maintenance database bd_test_alan y Username alan_admin. Introduce contraseña privadamente.

4. Guarda y despliega el servidor. Si falla, comprueba host interno, red y credenciales con el administrador.

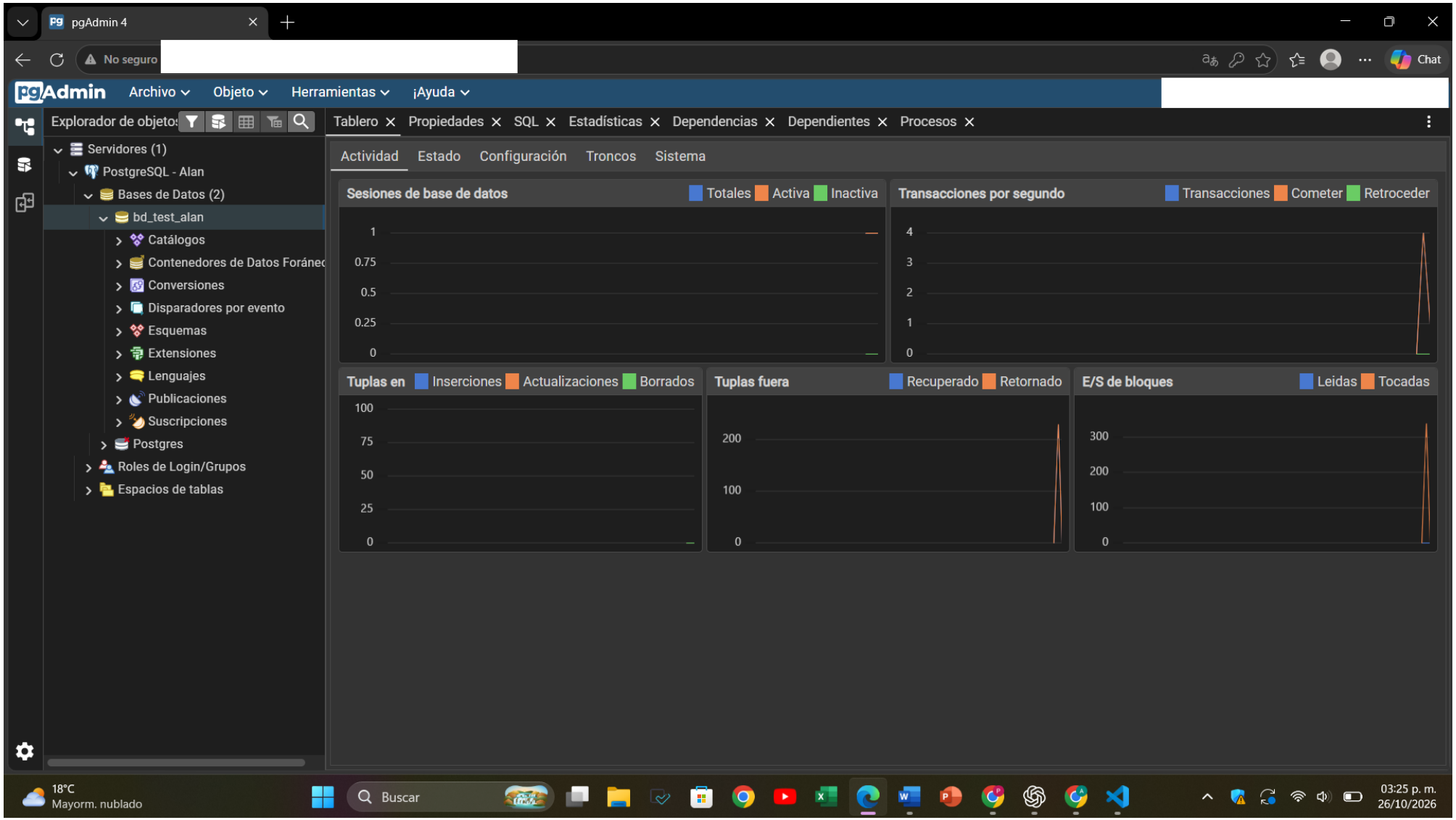
Figura 13. CAPS-06



Registro: puerto interno 5432 y base bd_test_alan. No confundir con el acceso externo de Windows.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 14. CAPS-07



Servidor registrado y conectado: el árbol permite identificar la base.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

5. Selecciona bd_test_alan, abre Tools > Query Tool y verifica identidad.

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT current_database(), current_user;
```

Espera bd_test_alan y tu cuenta efectiva. Si aparece otra base, selecciona la correcta antes de seguir. Continúa con el origen externo que exige la rúbrica.

9.3 Cliente PostgreSQL externo

La nota oficial exige consultas desde fuera de los contenedores, sin SSH/PuTTY para las pruebas. La web pgAdmin no acredita por sí sola todos los privilegios ni el origen externo del cliente SQL.

1. En Windows abre PowerShell con psql compatible instalado como cliente; no necesitas otro servidor.

2. Usa host aprobado y puerto externo. -W solicita contraseña sin incluirla en el comando.

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
psql -h HOST_VPS -p 40516 -U alan_admin -d bd_test_alan -W
```

3. Dentro de psql ejecuta SELECT current_database(), current_user; y el validador de permisos autorizado. -h es host, -p puerto, -U cuenta y -d base.

4. Toma I04 con ventana completa y reloj, sin secretos. SQL mediante docker exec o SSH no sustituye esa evidencia pendiente.

9.4 Cliente MySQL Workbench externo

Procedimiento preparado, sin captura externa válida. Dónde: Windows. Requisitos: Workbench y acceso autorizado.

1. Abre Workbench; en MySQL Connections pulsa +.

2. Etiqueta la conexión; elige Standard TCP/IP, hostname HOST_VPS, puerto 40517 y usuario alan_admin.

3. Pulsa Test Connection e introduce contraseña en el diálogo privado.

4. Abre el editor y verifica sesión.

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT DATABASE(), CURRENT_USER();

USE bd_test_alan;

SELECT DATABASE(), CURRENT_USER();
```

USE selecciona la base de esa sesión; no modifica tablas. Espera bd_test_alan y cuenta efectiva. Si falta acceso, pide revisión. Toma I05 cuando se compruebe. alan_admin no significa automáticamente Root o superadministración.

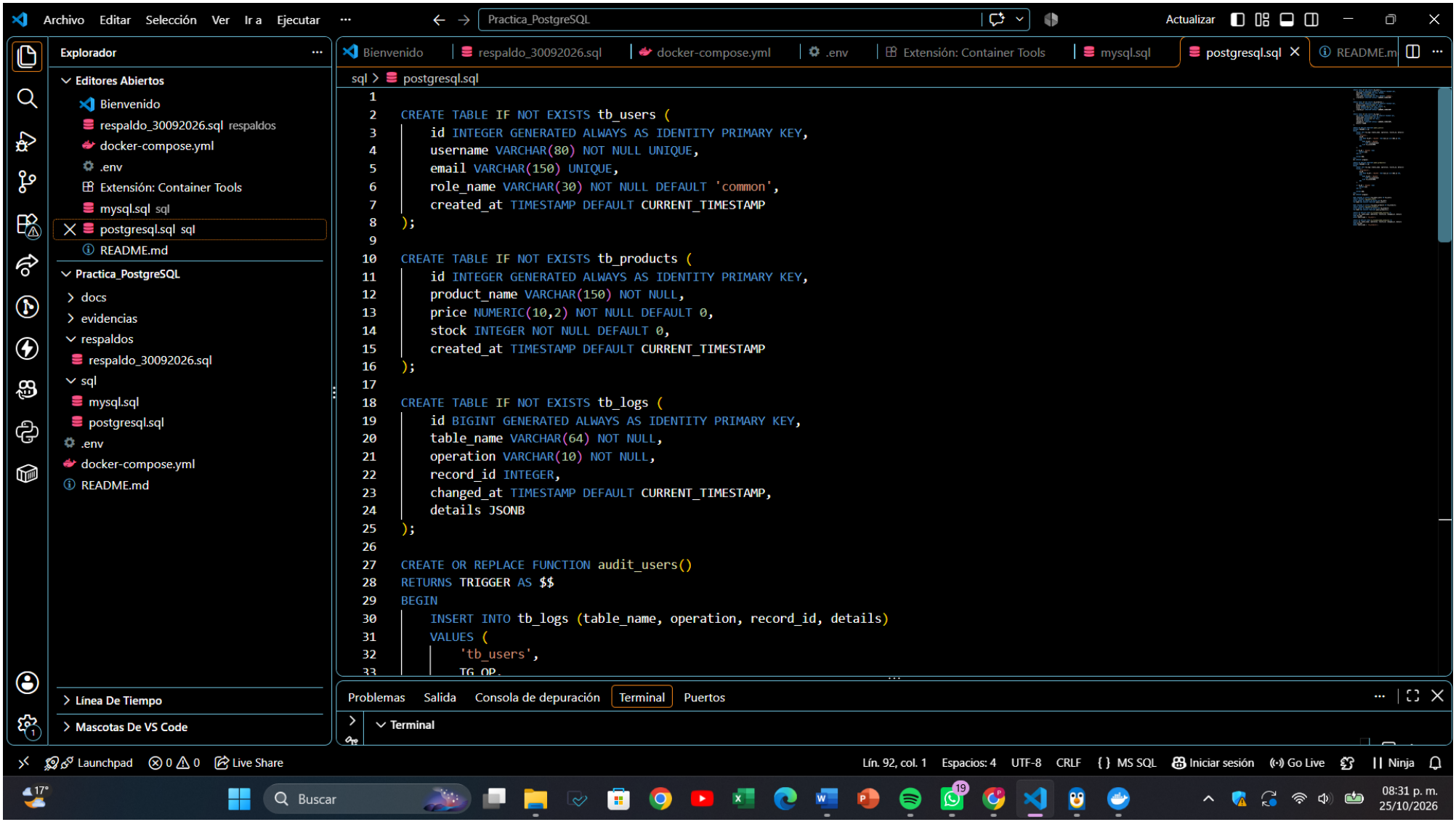
10. Bases, tablas y datos existentes

10.1 Leer antes de inicializar

Objetivo: reconocer objetos sin duplicar trabajo. Dónde: VS Code para archivos y cliente del motor para SQL. PostgreSQL incluye reemplazos de funciones/vistas y DROP TRIGGER; MySQL puede fallar al crear triggers. No reejecutes scripts completos sobre las bases activas.

En una base nueva aislada y autorizada, el responsable revisa primero sql/postgresql.sql o sql/mysql.sql, abre el archivo en el cliente correspondiente y ejecuta solo la inicialización aprobada. Mostrar código no demuestra ejecución. En el entorno existente continúa con SELECT y metadatos; cualquier diferencia requiere corrección mínima revisada.

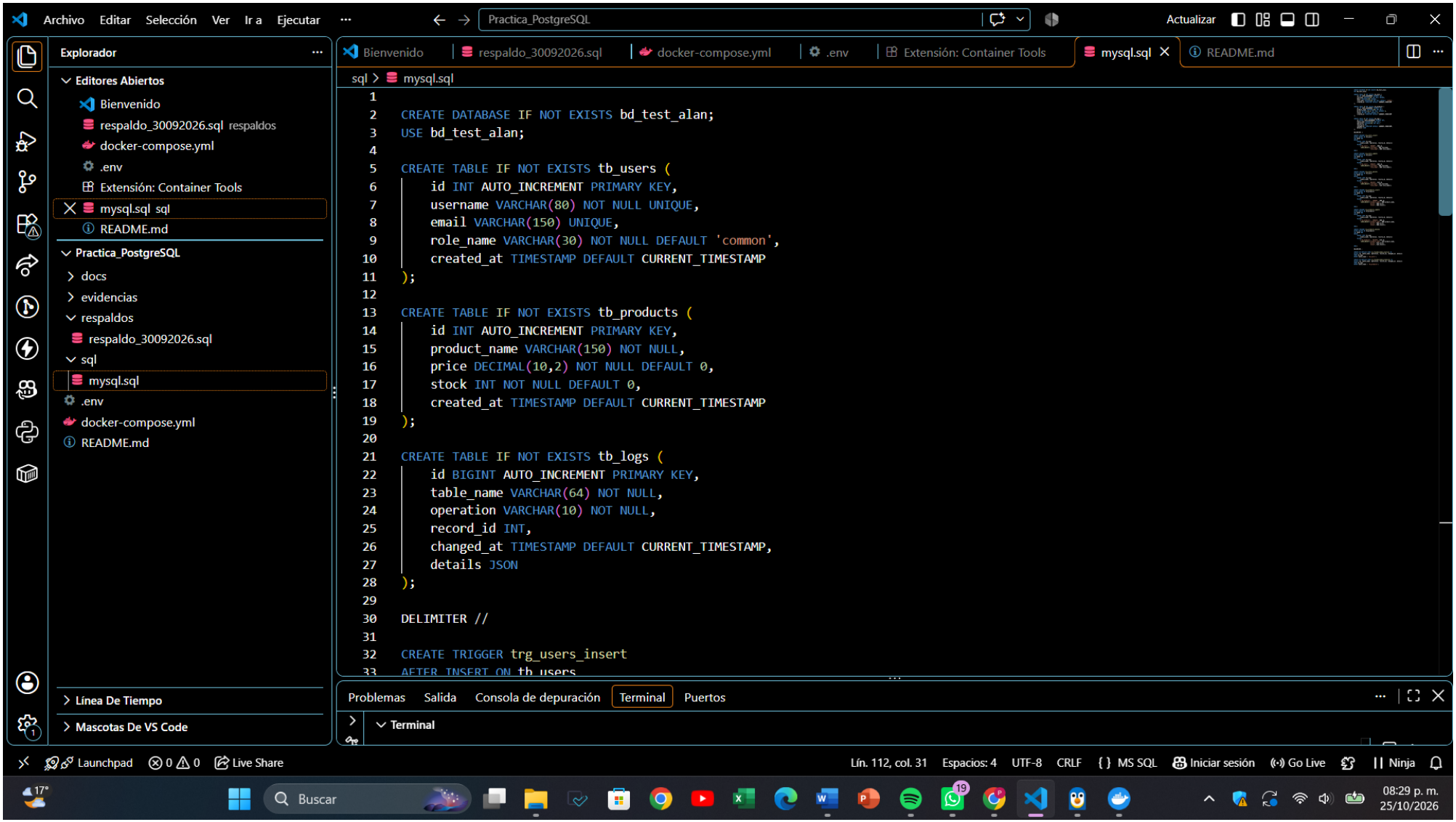
Figura 15. DOCX-22



Script PostgreSQL abierto: tablas y función de auditoría. No equivale a ejecución aprobada.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 16. DOCX-21



Script MySQL abierto: tablas e inicio de triggers. No repetir su importación completa.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

10.2 Entender la estructura

Tabla	Columnas principales	Restricciones declaradas
tb_users	id, username, email, role_name, created_at	PK; username obligatorio y único; email único admite NULL; rol por defecto common
tb_products	id, product_name, price, stock, created_at	PK; nombre obligatorio; precio (10,2) y stock entero con valor inicial 0
tb_logs	id, table_name, operation, record_id, changed_at, details	PK; referencia histórica a tabla/registro; JSON MySQL o JSONB PostgreSQL

PostgreSQL usa identidad; MySQL AUTO_INCREMENT. No se declaran FK para record_id ni CHECK de no negativos. Un eliminado puede seguir referenciado en el historial. Las restricciones de clase requieren cotejo docente: no se inventan reglas. Tres nombres de tablas adicionales en una foto no proporcionan su diseño oficial.

10.3 Localizar y verificar tablas

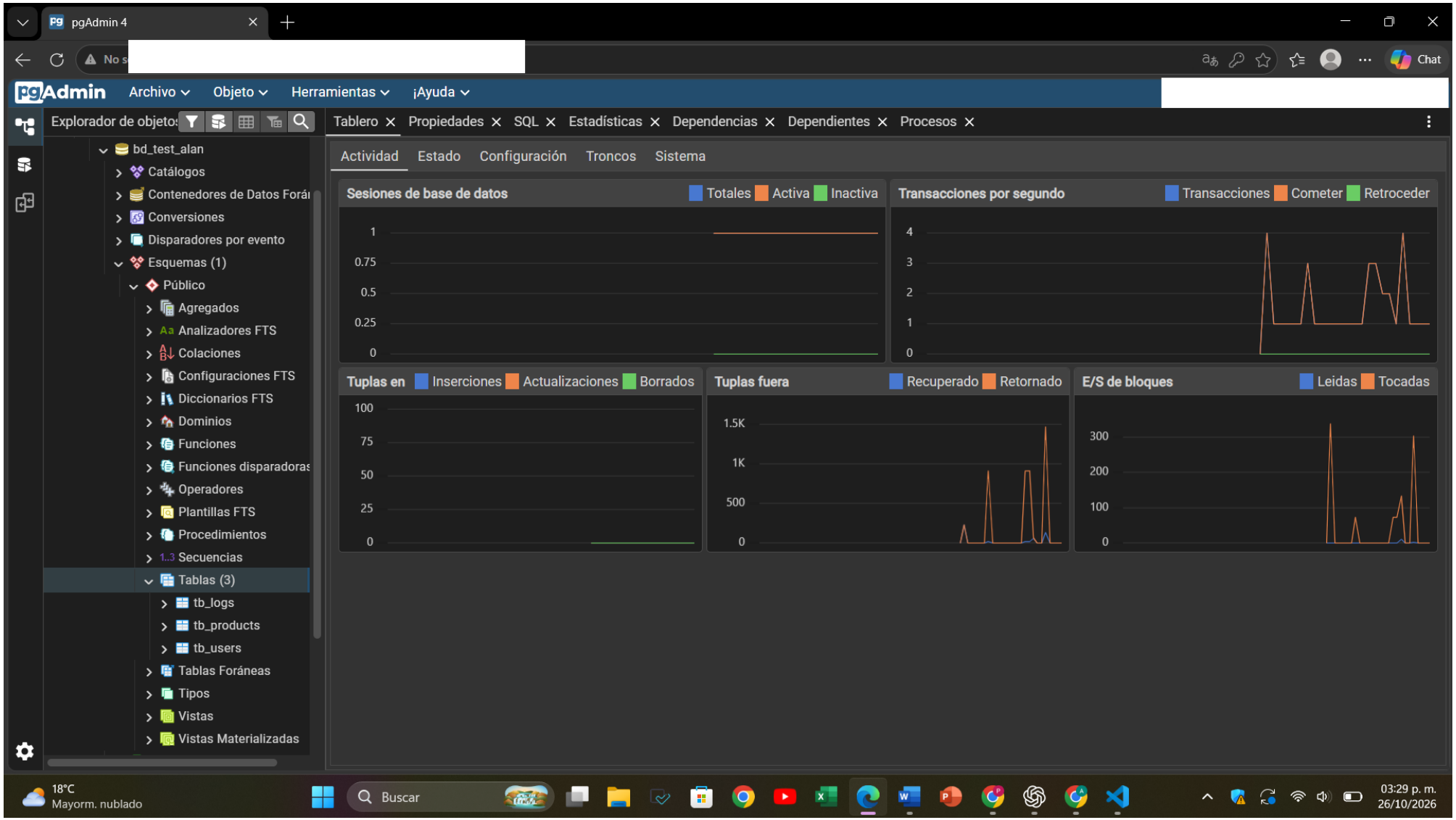
1. En pgAdmin despliega Servers > tu servidor > Databases > bd_test_alan > Schemas > public > Tables. Usa Refresh si falta actualización, no recrees objetos.

2. En Query Tool consulta:

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE'
ORDER BY table_name;
```

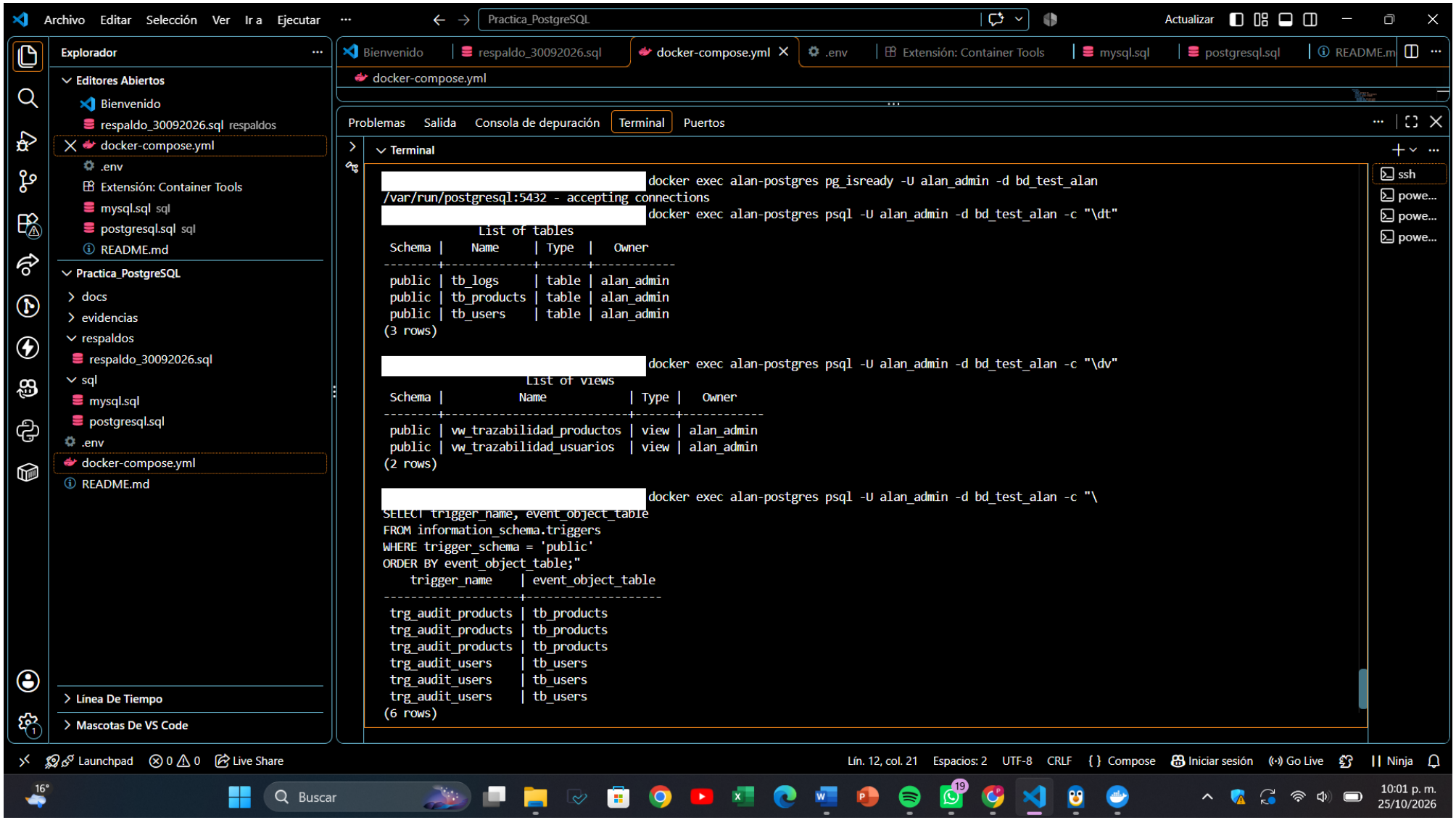
Figura 17. CAPS-08



Tres tablas en el árbol PostgreSQL. Existencia no acredita todos sus permisos o restricciones.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 18. DOCX-32



Listados históricos de tres tablas, dos vistas y eventos de auditoría. Consulta interna, no cliente externo.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

3. En Workbench, seleccionada bd_test_alan, revisa sin cambiar:

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SHOW FULL TABLES FROM bd_test_alan;

SHOW CREATE TABLE bd_test_alan.tb_users;

SHOW CREATE TABLE bd_test_alan.tb_products;

SHOW CREATE TABLE bd_test_alan.tb_logs;
```

Espera definiciones coherentes con los archivos. Documenta diferencias antes de corregir. Los validadores consultan columnas e índices; no se ejecutaron contra el servidor en esta revisión.

10.4 Contar sin escribir

Dónde: Query Tool o Workbench en bd_test_alan. Consulta común a ambos motores:

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

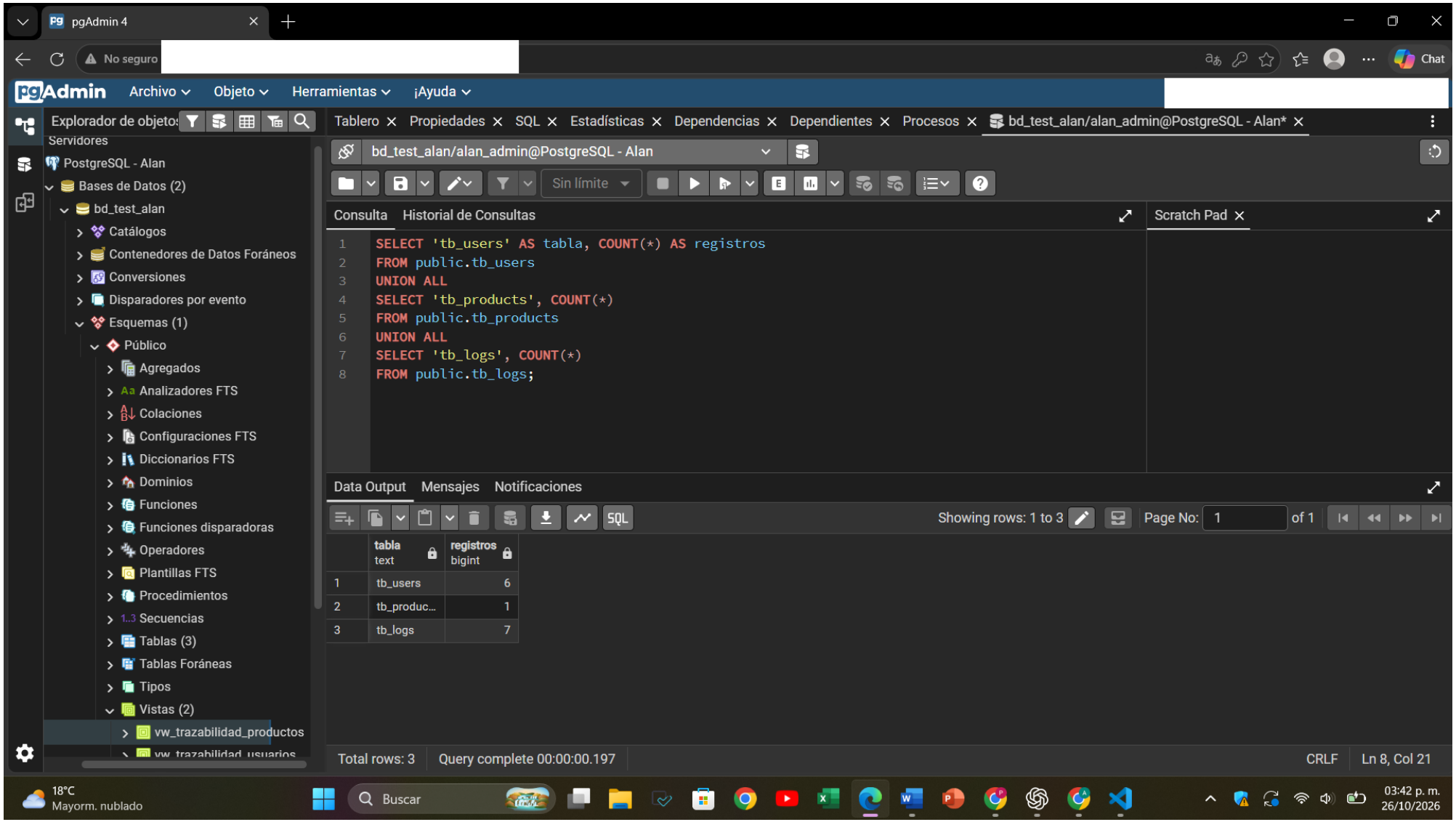
```
SELECT 'tb_users' AS tabla, COUNT(*) AS registros FROM tb_users

UNION ALL SELECT 'tb_products', COUNT(*) FROM tb_products

UNION ALL SELECT 'tb_logs', COUNT(*) FROM tb_logs;
```

Se observaron 6/1/7 en PostgreSQL antes de los UPDATE posteriores y 5/4/0 en MySQL. No son valores actuales certificados. No repitas INSERT ni fuerces cantidades.

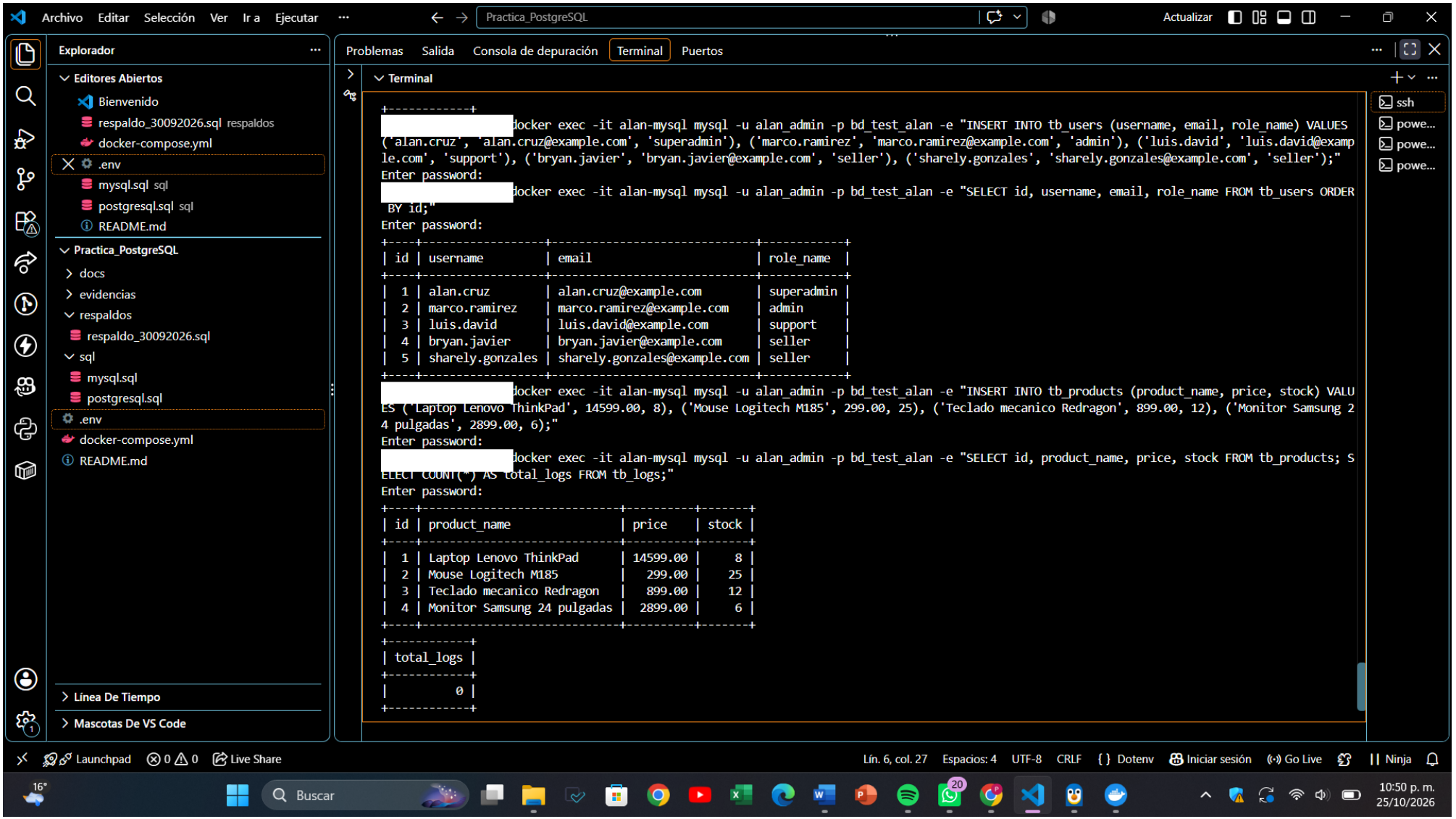
Figura 19. CAPS-14



Conteos PostgreSQL históricos: 6 usuarios, 1 producto, 7 logs antes de las actualizaciones posteriores.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 20. DOCX-35



Usuarios y cuatro productos MySQL con cero logs. Los INSERT visibles son históricos: no repetirlos.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

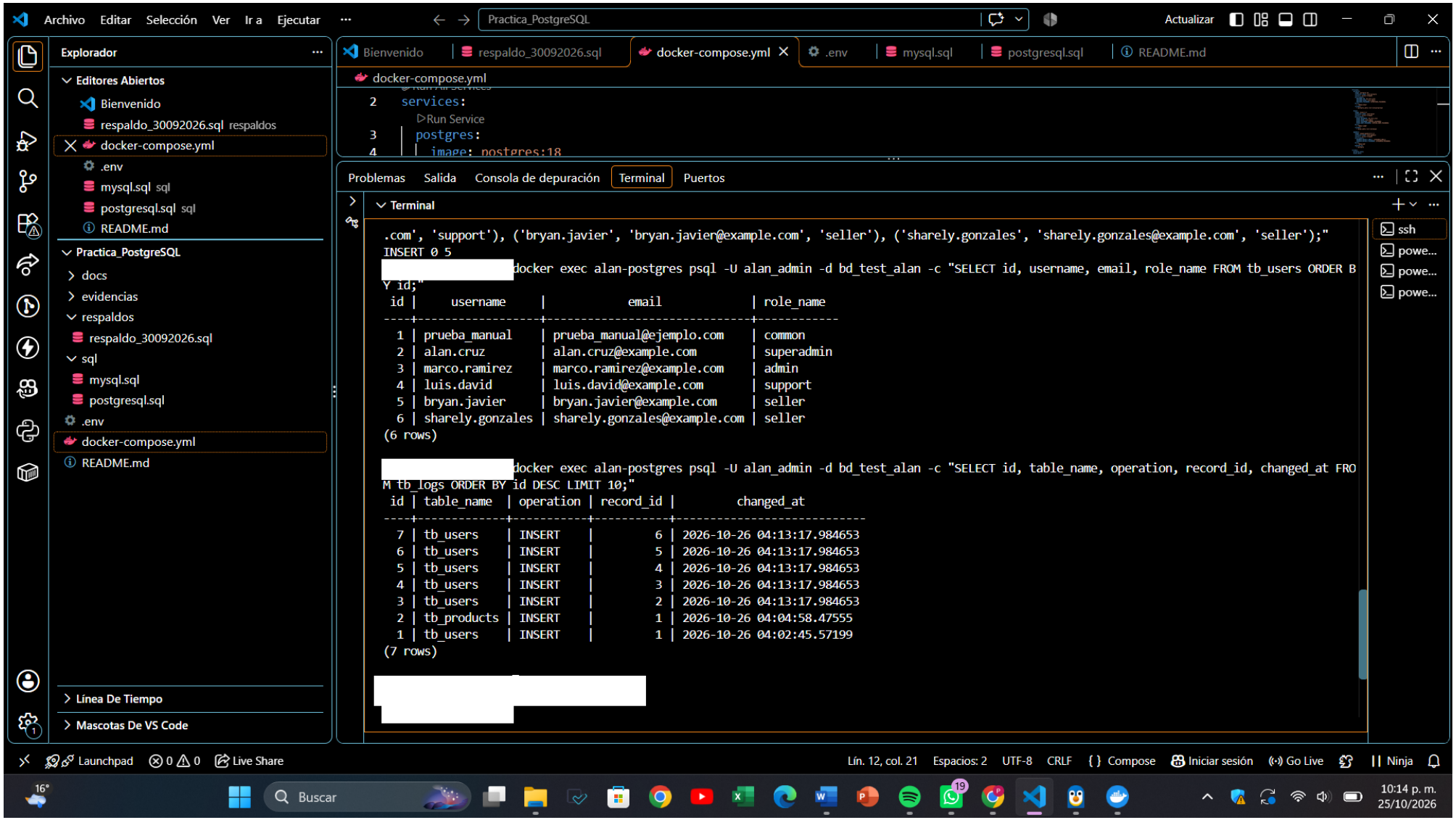
11. Usuarios, roles y privilegios

11.1 Filas y cuentas son distintas

tb_users contiene datos de aplicación; role_name es texto. No crea una cuenta ni concede permisos. alan_admin identifica una conexión, no prueba privilegios globales.

La rúbrica exige siete roles reales por motor: superadmin, admin, seller, buyer, support, common y not_registered. Las cinco identidades del proyecto son alan.cruz/superadmin, marco.ramirez/admin, luis.david/support, bryan.javier/seller y sharely.gonzales/seller. Las filas se observan; faltan cuentas reales, membresías y privilegios. Sharely es una persona, no dos integrantes inventados.

Figura 21. DOCX-34



Filas PostgreSQL y logs de inserción. Incluyen prueba_manual; no son el catálogo de cuentas del motor.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

11.2 Comprobar permisos efectivos

Objetivo: verificar autorización real. Dónde: cliente externo conectado como el actor evaluado.

1. PostgreSQL: SELECT current_database(), session_user, current_user; identifica base y rol efectivo.

2. Revisa sql/validacion_permisos_postgresql.sql: lecturas de pg_roles, membresías y privilegios de tablas.

3. MySQL: consulta sesión y concesiones.

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT DATABASE(), USER(), CURRENT_USER(), CURRENT_ROLE();
```

```
SHOW GRANTS FOR CURRENT_USER;
```

4. Usa sql/validacion_permisos_mysql.sql para los permisos visibles. Si se deniega una consulta, registra el rechazo sin elevar privilegios.

5. Asocia evidencia a I06-I08, sin hashes o tablas de autenticación.

Espera roles, cuentas, membresías y permisos comprobables, no inferidos. Los SQL del manual no contienen CREATE ROLE/USER ni GRANT: tenerlos no resuelve esos indicadores.

11.3 Diferencias con Práctica 02

Support tiene SELECT/INSERT/UPDATE de usuarios y SELECT/UPDATE de productos en la referencia, sin DELETE. Seller tiene SELECT/INSERT/UPDATE de productos. La foto pide que Support elimine: si se conserva esa política, debe denegarse. El profesor debe aclarar si se evalúa el rechazo o una política diferente antes de cambiar permisos.

Root no equivale automáticamente a alan.cruz o alan_admin; sin rol no equivale a common. Un comentario anterior sobre CRUD sin rol concede acceso a una cuenta que también aparece como superadmin: no acredita una cuenta independiente. Se conserva el antecedente sin copiar contraseñas ni cambiar prácticas previas. Continúa con auditoría histórica.

12. Auditoría y vistas de trazabilidad

12.1 Seguir una operación

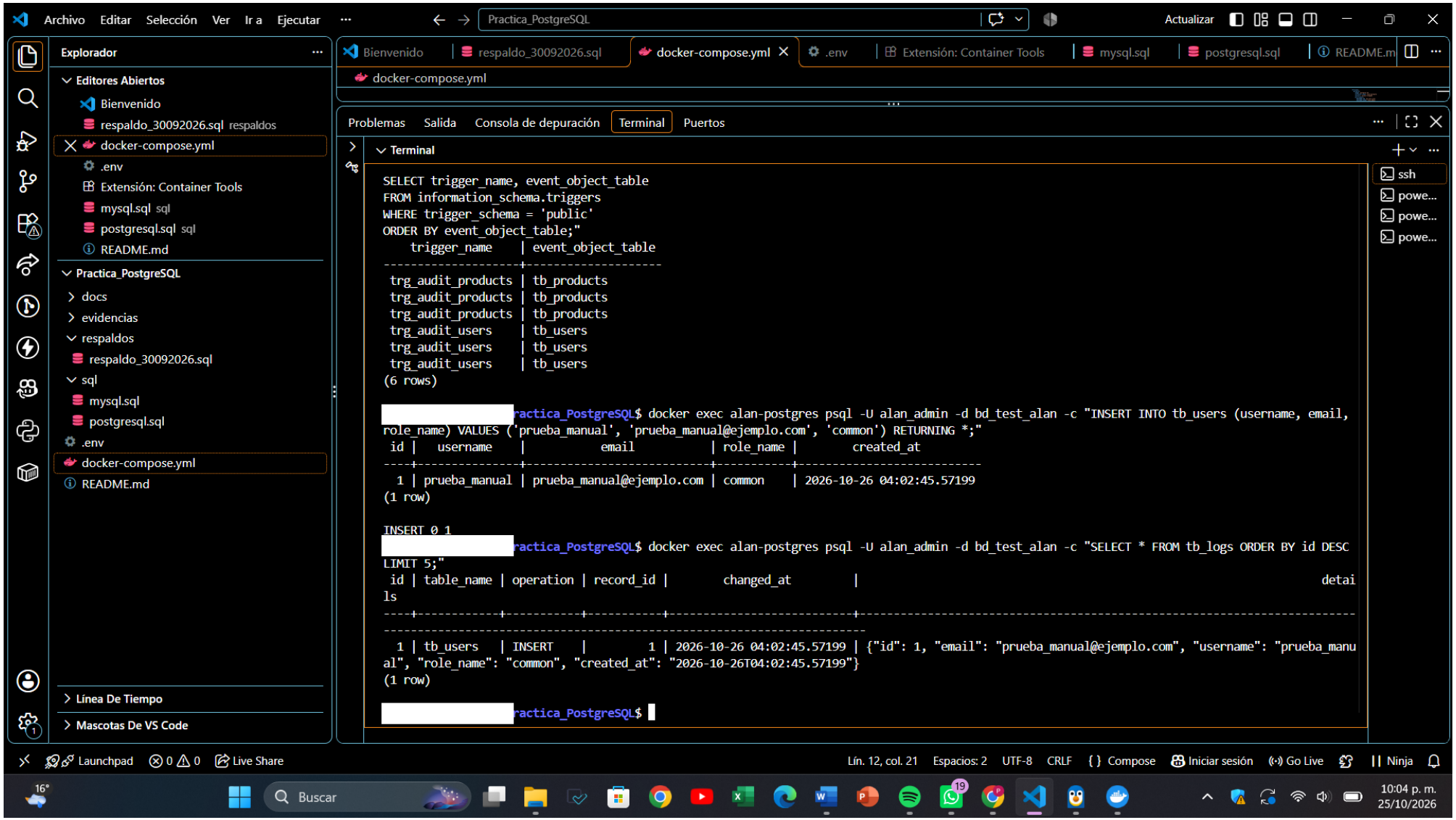
Un trigger AFTER se activa tras una operación permitida y añade un evento a tb_logs. La vista consulta el historial, no lo duplica. SELECT no debería aumentar logs.

PostgreSQL declara dos triggers (uno por tabla), cada uno para INSERT/UPDATE/DELETE, y dos funciones. information_schema.triggers muestra seis filas de eventos, no seis objetos distintos. MySQL declara seis triggers separados, pero falta instalación comprobada. No se importa la cifra de otra práctica como requisito.

12.2 Inserción histórica auditada

La imagen muestra prueba_manual y su log. Explica el mecanismo sin autorizar repetirlo. No acredita las cuotas ni los actores oficiales.

Figura 22. DOCX-33



INSERT histórico PostgreSQL y evento asociado. No repetir la inserción.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Consulta lo existente sin escribir:

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT id, table_name, operation, record_id, changed_at, details
FROM tb_logs ORDER BY id DESC;
```

id identifica el evento y record_id la entidad. Un DELETE puede referirse a una fila inexistente actualmente; no exijas INNER JOIN para reconstruirla.

12.3 Verificar eventos PostgreSQL

1. En Query Tool de bd_test_alan ejecuta:

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

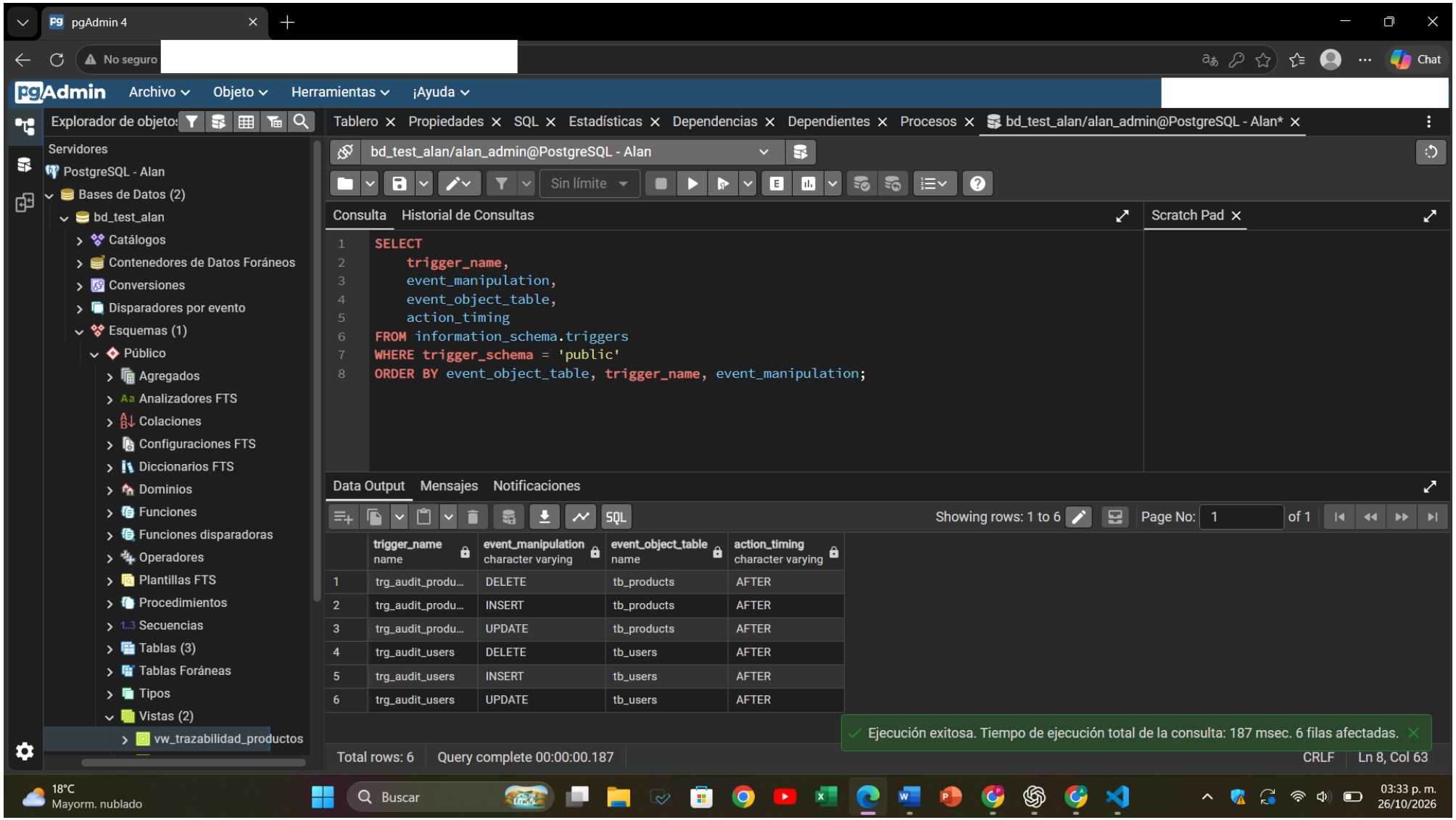
```
SELECT trigger_name, event_manipulation, event_object_table
FROM information_schema.triggers WHERE trigger_schema = 'public'
ORDER BY event_object_table, trigger_name, event_manipulation;
```

2. Espera dos nombres repetidos por tres eventos. Para contar objetos sin internos:

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT tgname FROM pg_trigger WHERE NOT tgisinternal ORDER BY tgname;
```

Figura 23. CAPS-11



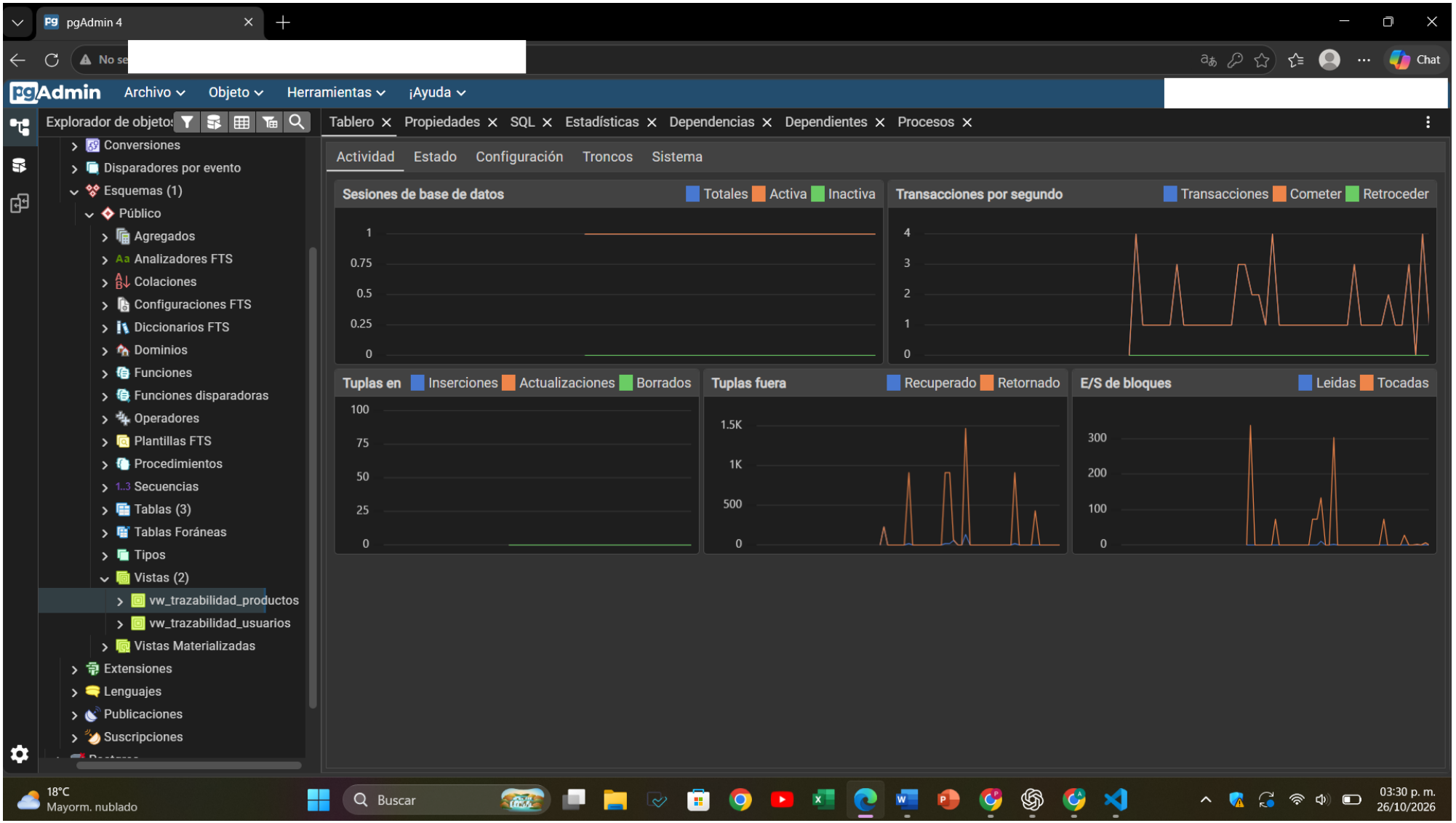
Seis filas de eventos para dos triggers. Hay INSERT/UPDATE históricos; no se certifica DELETE no demostrado.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

12.4 Consultar vistas

1. Despliega Schemas > public > Views; usa Refresh si es necesario.

Figura 24. CAPS-09



Dos vistas en el árbol. Su existencia se complementa con consultas de resultados.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

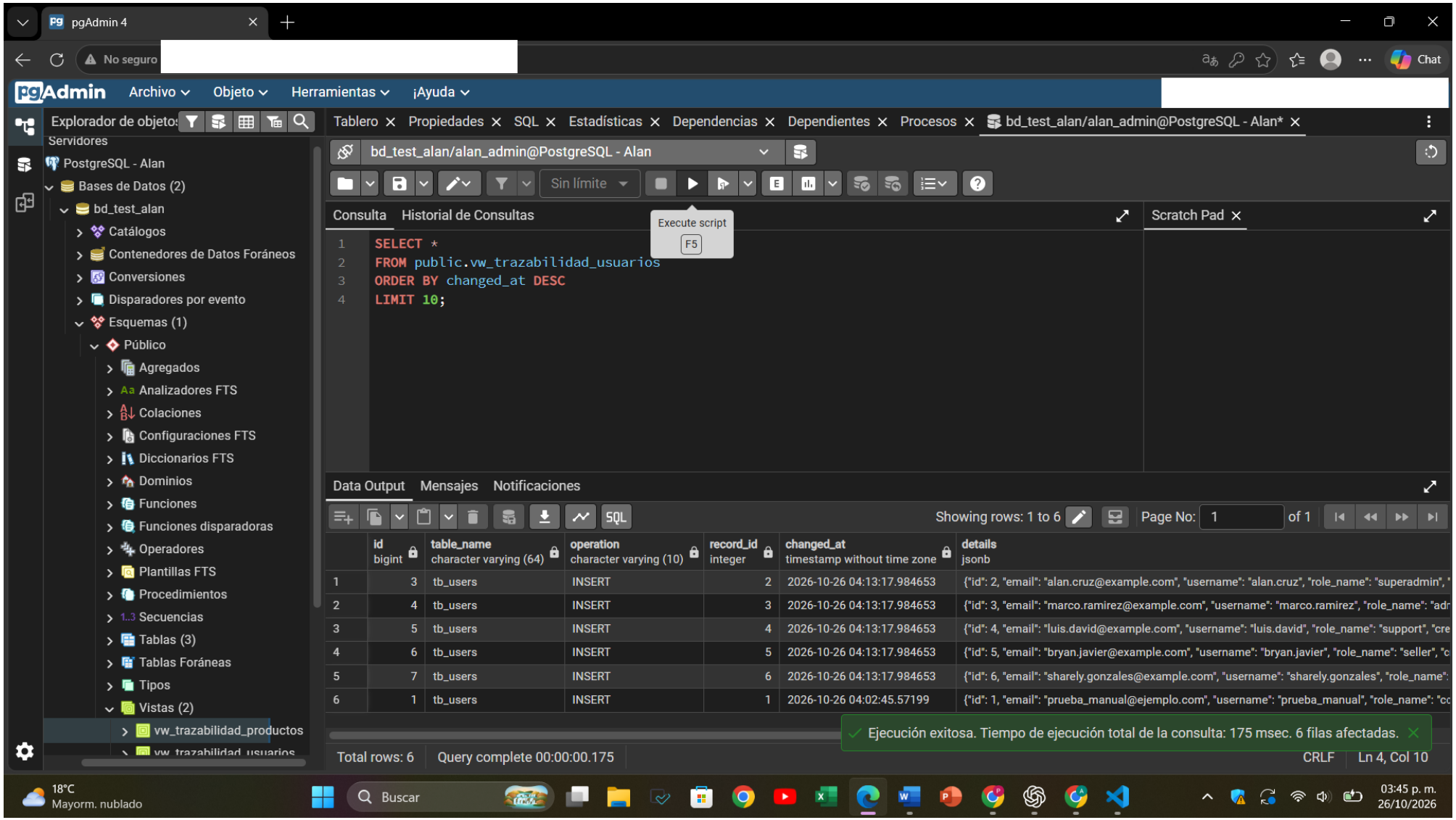
2. En Query Tool consulta el historial ordenado, sin DISTINCT para ocultar fallos.

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT * FROM public.vw_trazabilidad_usuarios ORDER BY changed_at, id;
```

```
SELECT * FROM public.vw_trazabilidad_productos ORDER BY changed_at, id;
```

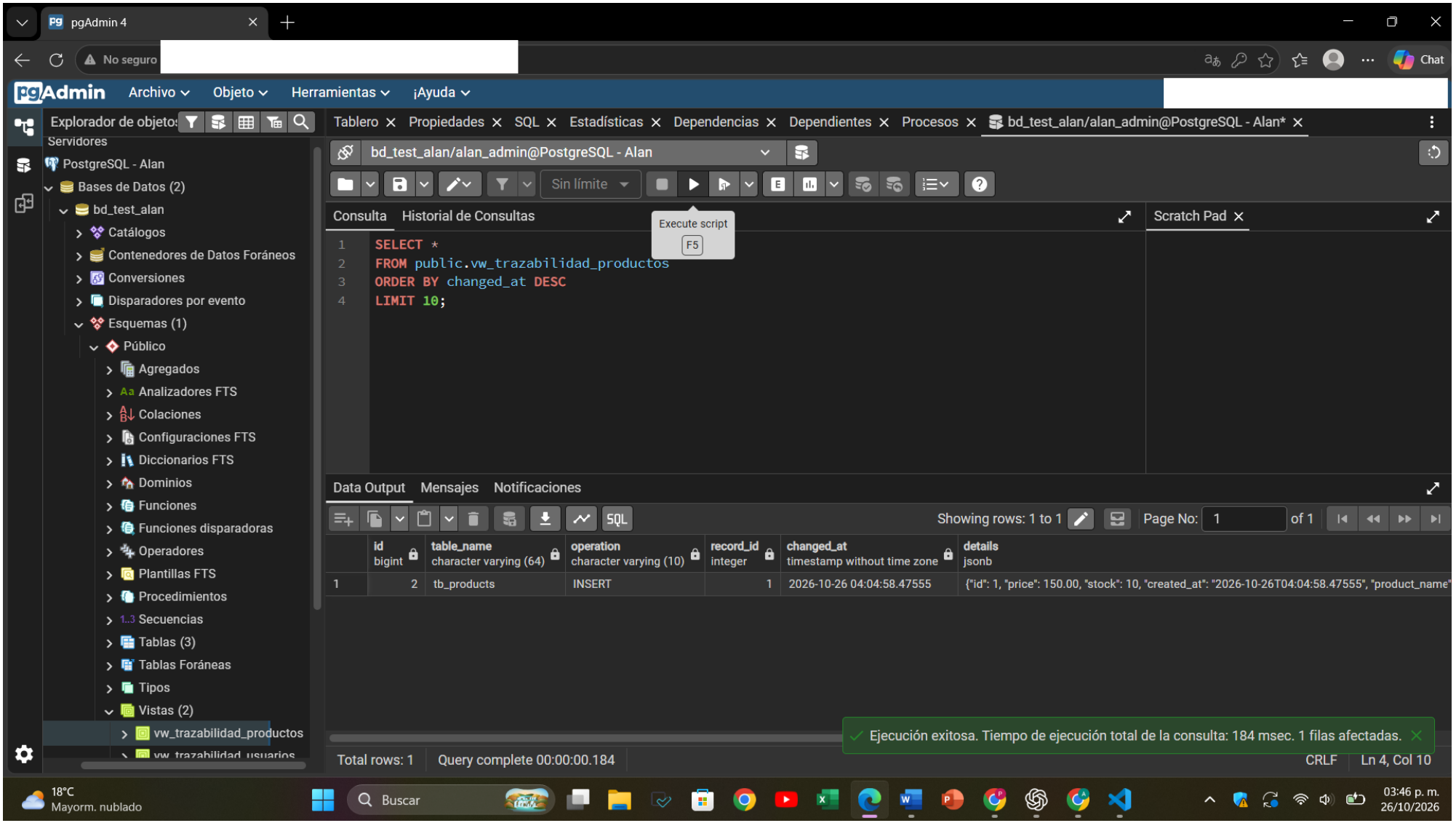
Figura 25. CAPS-15



Vista de usuarios con INSERT históricos. No demuestra permisos de cuentas.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 26. CAPS-16



Vista de productos con INSERT histórico. No acredita todos los ejercicios.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

3. Compara cada vista con tb_logs filtrada por table_name: debe representar el mismo historial.

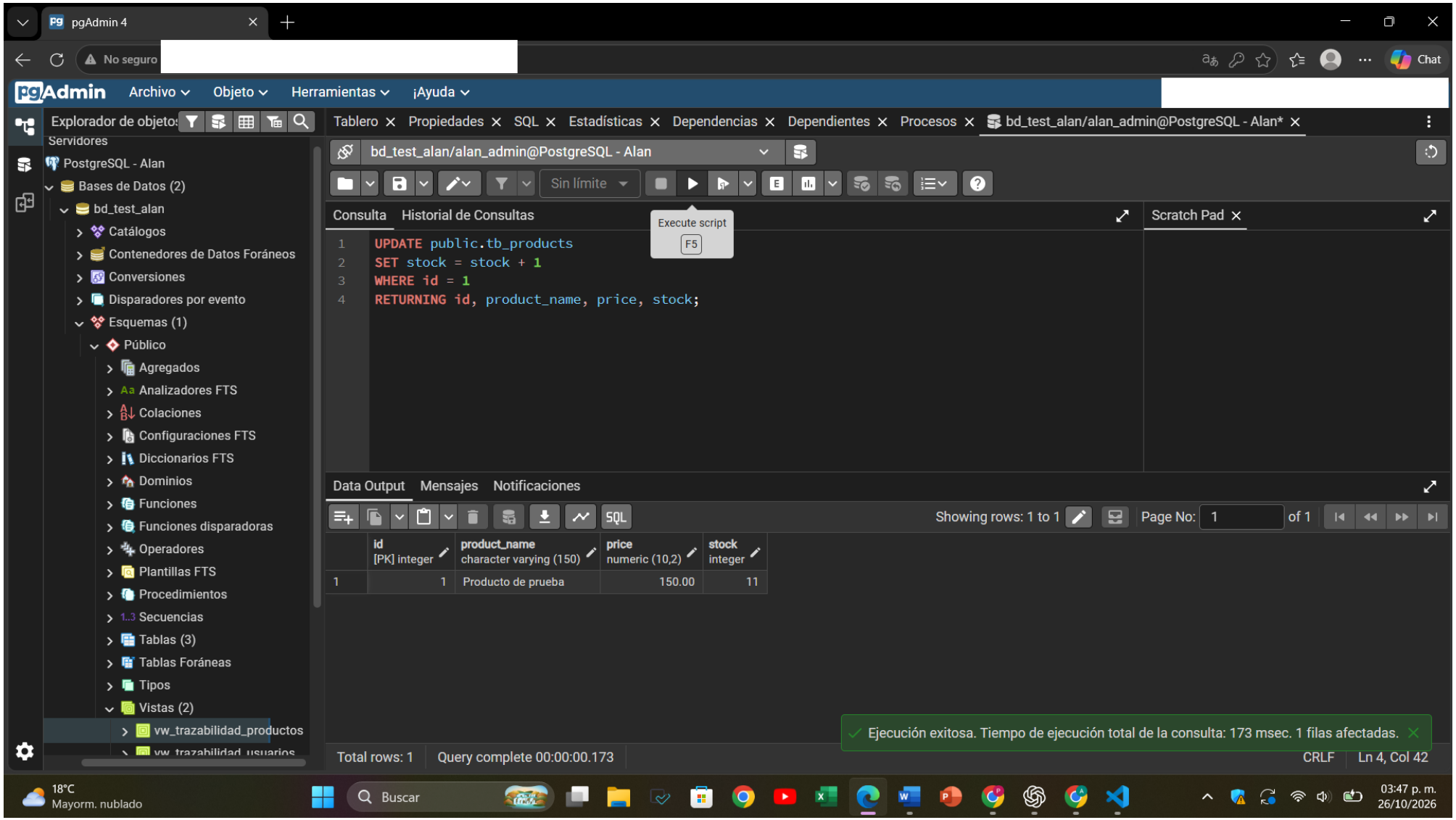
4. Revisa definiciones con los validadores. No hay actor explícito en estas bitácoras. role_name en details describe al afectado, no necesariamente a quien ejecutó la acción.

12.5 Actualizaciones ya realizadas

Las parejas siguientes muestran operación y evento. No vuelvas a ejecutar los UPDATE impresos. sql/validacion_historial_postgresql.sql permite consultar lo existente sin escribir.

PostgreSQL guarda NEW en INSERT/UPDATE y OLD en DELETE; no OLD y NEW juntos por UPDATE. MySQL preparado utiliza JSON parcial. No se promete autoría o detalle que el diseño no registra. Cualquier requisito adicional necesita propuesta separada.

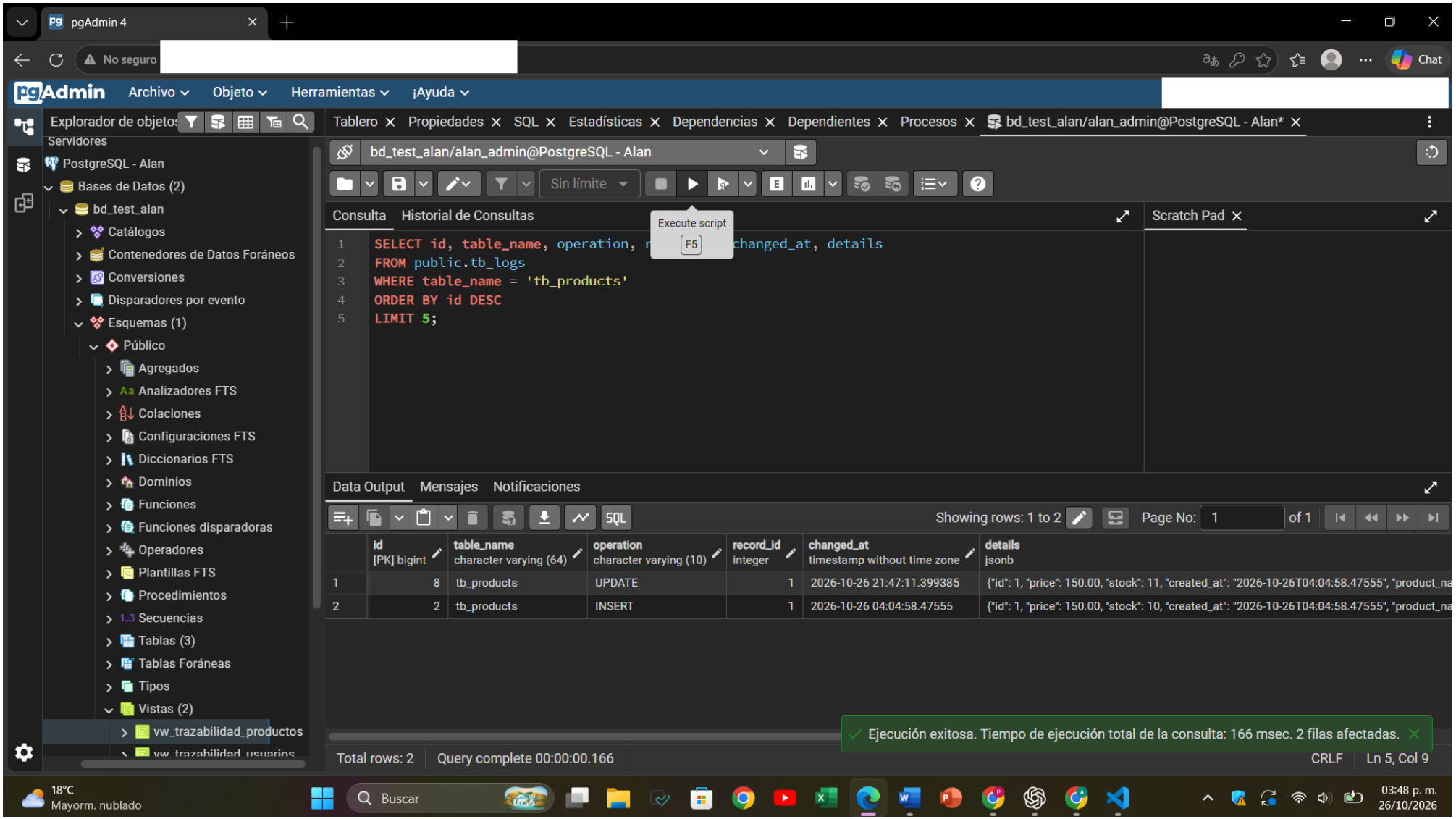
Figura 27. CAPS-17



UPDATE histórico del producto 1 con stock 11. No es un paso pendiente.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

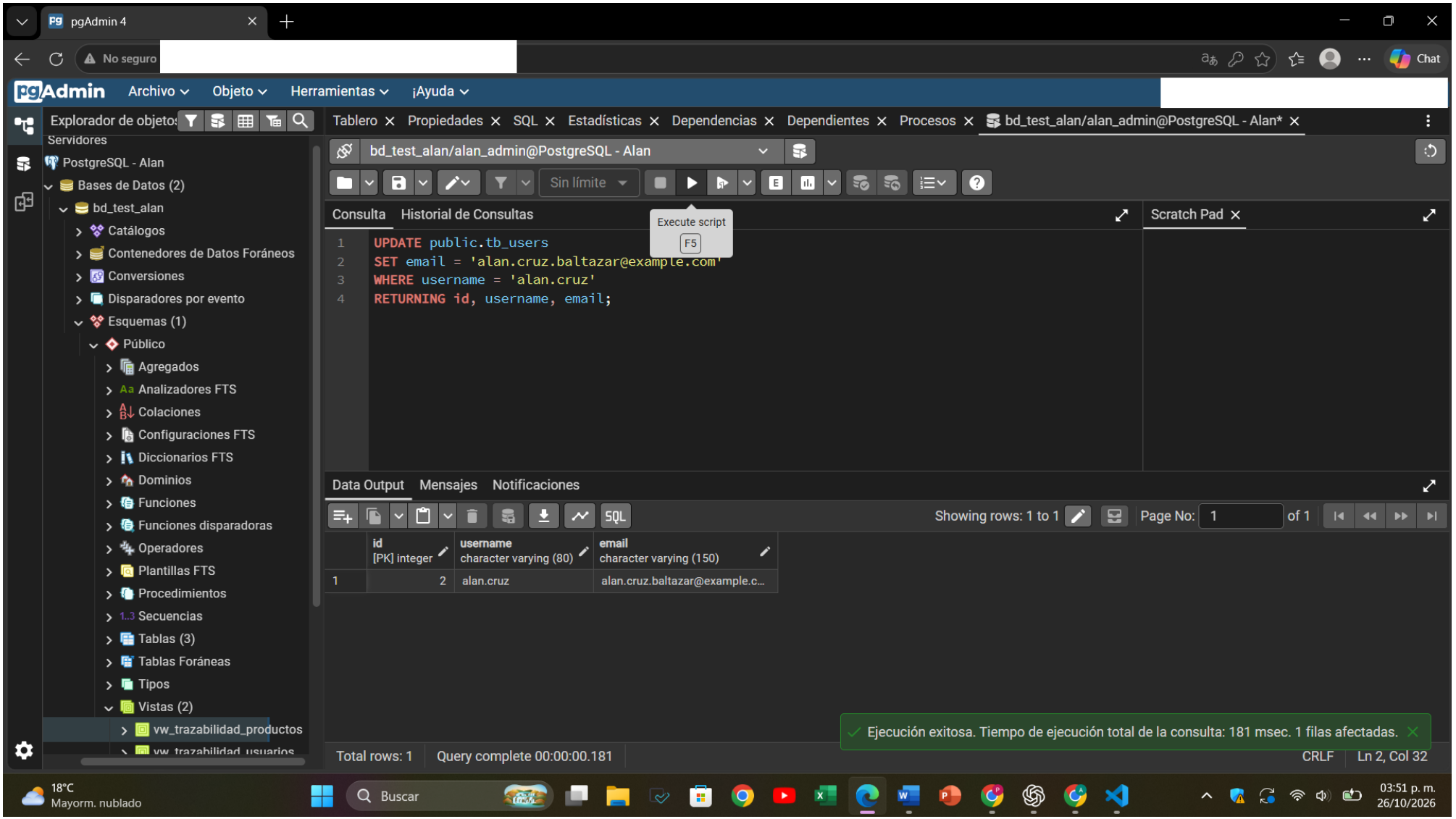
Figura 28. CAPS-18



Log UPDATE del producto y su INSERT anterior: estados guardados en eventos distintos.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

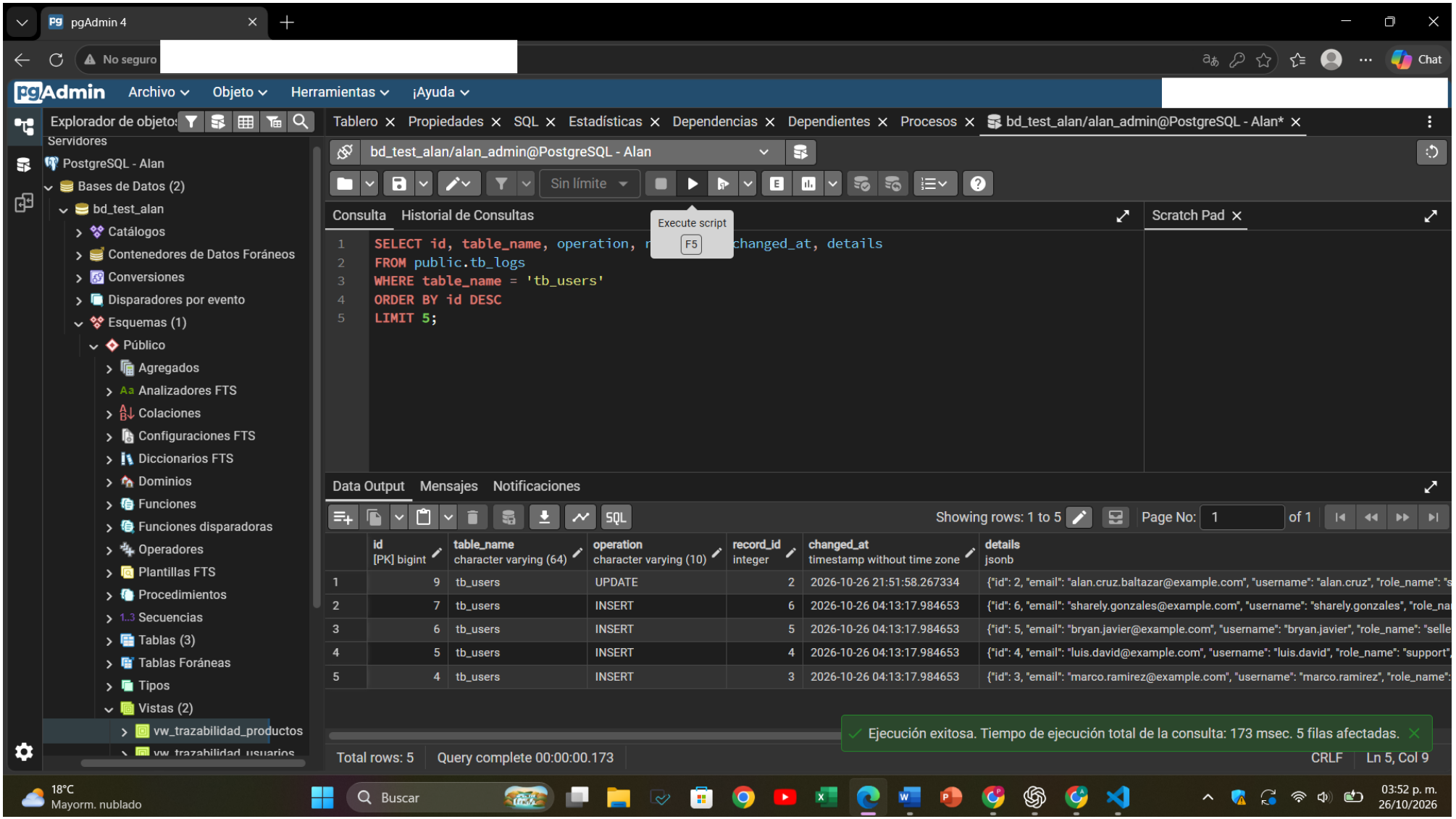
Figura 29. CAPS-19



UPDATE histórico del correo de alan.cruz con una fila devuelta. No acredita identidad del actor personal.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

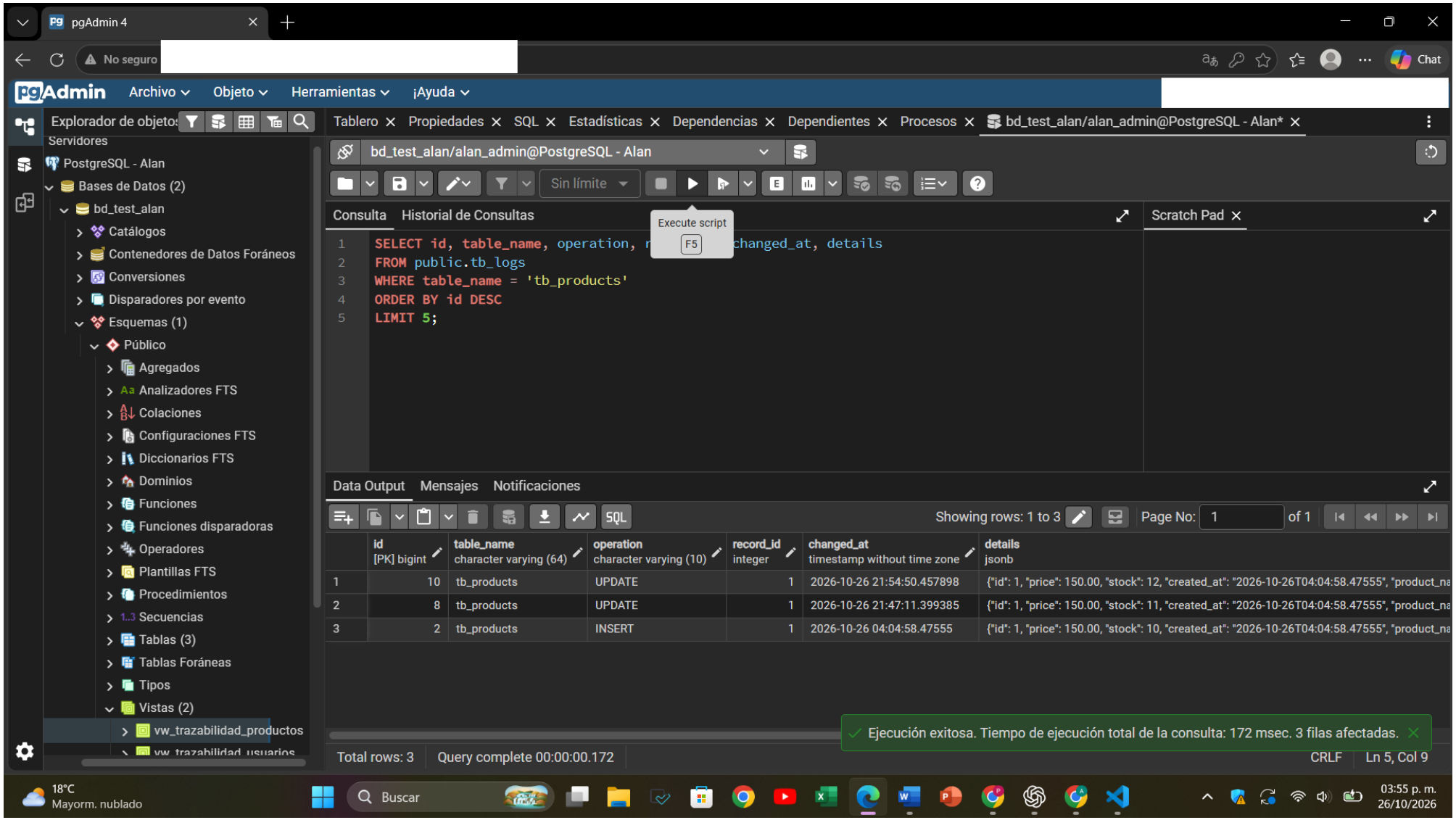
Figura 30. CAPS-20



Log UPDATE del usuario afectado. ID del evento y record_id tienen funciones distintas.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

Figura 31. CAPS-22



UPDATE a stock 11 y luego 12 son eventos distintos, no duplicados. El ID máximo no es COUNT(*).

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

12.6 MySQL pendiente

SHOW TRIGGERS no mostró objetos, la confianza seguía OFF y los logs estaban vacíos. Las vistas se declaran en SQL, pero no se certifica instalación ni resultados. El capítulo 15 explica la continuación con el administrador sin eludir permisos ni repetir importación.

13. Ocho apartados y diez casos oficiales

13.1 Conservar la fuente real

La foto enumera ocho apartados para ambos motores. Los indicadores 38-47 exigen diez casos de clase, sin enunciarlos. No se encontraron los casos 9/10 en los materiales revisados. No se dividen artificialmente actividades combinadas.

Apartado	Actividad manuscrita	Estado real
E01	Root inserta 3 usuarios	Pendiente por actor/motor
E02	Sin rol inserta 3 usuarios	Falta identificar cuenta y privilegios
E03	Support inserta 3 usuarios	Falta rol efectivo y eventos
E04	Root modifica otro usuario y elimina uno	Falta actor y DELETE comprobados
E05	Sin rol modifica y elimina uno	Depende de privilegios confirmados
E06	Support modifica y elimina uno	Aclarar si DELETE debe denegarse
E07	Root inserta 2 productos	Sin acreditación completa por actor
E08	Vendedor 1 inserta 5; vendedor 2 inserta 5000 productos	Alcance por confirmar; no ejecutado

Los 5000 pertenecen a E08, no a un indicador independiente. No se cargaron. Los datos existentes impiden asumir un inicio vacío o forzar 5007. Los tres nombres de tablas al pie requieren definición oficial antes de crearlas.

13.2 Cómo documentar después de autorizar

1. Confirma casos 9/10, identidad Root/sin rol y DELETE Support con el profesor.

2. Desde cliente externo identifica motor, base, actor y permisos reales.

3. Revisa historial para no repetir operaciones demostradas.

4. Solo con autorización registra SQL exacto, resultado esperado/observado, conteos antes/después y log o denegación. No ejecutes cargas masivas desde la guía.

5. CASOS_SIMULACION.md/.csv tiene una fila por apartado/motor con objetivo, actor/rol, SQL propuesto, expectativa, resultado y evidencia. Una plantilla no es prueba aprobada.

Estado actual: INSERT/UPDATE PostgreSQL históricos, sin acreditar toda la matriz por actor; MySQL bloqueado. Indicador 37 requiere constancia de entrega segura de accesos sin mostrar credenciales.

14. Respaldos completos de ambos motores

14.1 Reconocer un respaldo

Objetivo: exportar privadamente estructura/datos de bd_test_alan y sus objetos existentes. SQL de creación o captura no equivalen a respaldo completo. La fotografía pide ambos motores.

Solo se encontró respaldos/respaldo_30092026.sql, de 2 970 514 bytes, tres tablas y marcador final. No demuestra el VPS actual ni restaurabilidad. No se encontró respaldo PostgreSQL de datos. No se exportó nada en esta sesión.

14.2 Procedimiento preparado

Requisitos: clientes compatibles (pg_dump/pg_restore 18 y mysqldump 8.4), red y permisos autorizados. Dónde: PowerShell Windows. RESPALDOS_PREPARACION.md contiene los bloques completos con destinos nuevos, comprobaciones y errores; no ejecutados.

1. Usa el bloque completo para crear carpeta privada nueva con GUID fuera de Git, sin sobrescribir.

2. Introduce servidor mediante Read-Host. La contraseña la solicita el cliente; no va en comandos.

3. El bloque MySQL prepara variables y exporta con:

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
$ErrorActionPreference = 'Stop'
$destino = Join-Path $env:USERPROFILE ('Documents/Respaldos_privados_BDN/' +
  [guid]::NewGuid().ToString('N'))
if (Test-Path -LiteralPath $destino) { throw 'El destino ya existe' }
New-Item -ItemType Directory -Path $destino | Out-Null
$servidorBD = Read-Host 'Servidor autorizado (no compartir capturas de este dato)'
$archivoMysql = Join-Path $destino 'bd_test_alan_mysql.sql'
& mysqldump --host=$servidorBD --port=40517 --user=alan_admin --password
  --single-transaction --routines --events --triggers --hex-blob --no-tablespaces
  --set-gtid-purged=OFF --databases bd_test_alan --result-file=$archivoMysql
if ($LASTEXITCODE -ne 0) { throw 'Exportación MySQL falló; conservar archivo parcial sin
   declararlo válido' }
if ((Get-Item -LiteralPath $archivoMysql).Length -eq 0) { throw 'Archivo vacío' }
Get-Item -LiteralPath $archivoMysql | Select-Object Name,Length
Get-FileHash -LiteralPath $archivoMysql -Algorithm SHA256
```

La línea larga continúa visualmente. Copia el comando completo desde el archivo fuente; no agregues saltos de línea.

--single-transaction cubre tablas transaccionales sin DDL concurrente. Rutinas/eventos/triggers requieren privilegios y exportan solo los existentes. No publiques el SQL ni uses redirecciones que cambien codificación.

4. El bloque PostgreSQL prepara destino, exporta custom y lee catálogo:

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
$ErrorActionPreference = 'Stop'
$destino = Join-Path $env:USERPROFILE ('Documents/Respaldos_privados_BDN/' +
```

```
  [guid]::NewGuid().ToString('N'))
if (Test-Path -LiteralPath $destino) { throw 'El destino ya existe' }
New-Item -ItemType Directory -Path $destino | Out-Null
$servidorBD = Read-Host 'Servidor autorizado (no compartir capturas de este dato)'
$archivoPg = Join-Path $destino 'bd_test_alan_postgresql.dump'
& pg_dump --host=$servidorBD --port=40516 --username=alan_admin --password
  --dbname=bd_test_alan --format=custom --create --file=$archivoPg
if ($LASTEXITCODE -ne 0) { throw 'Exportación PostgreSQL falló; conservar archivo
  parcial sin declararlo válido' }
if ((Get-Item -LiteralPath $archivoPg).Length -eq 0) { throw 'Archivo vacío' }
$inventarioPg = Join-Path $destino 'inventario_pg_restore.txt'
& pg_restore --list --file=$inventarioPg $archivoPg
if ($LASTEXITCODE -ne 0) { throw 'No se pudo leer el catálogo del respaldo' }
Get-Item -LiteralPath $archivoPg | Select-Object Name,Length
Get-FileHash -LiteralPath $archivoPg -Algorithm SHA256
```

La línea larga continúa visualmente. Copia el comando completo desde el archivo fuente; no agregues saltos de línea.

5. Comprueba salida exitosa, archivo no vacío, tamaño, fecha, SHA-256 e inventario. Captura metadatos sin contenido privado, I11. Si falla, conserva el parcial sin declararlo válido.

14.3 Límites de verificación

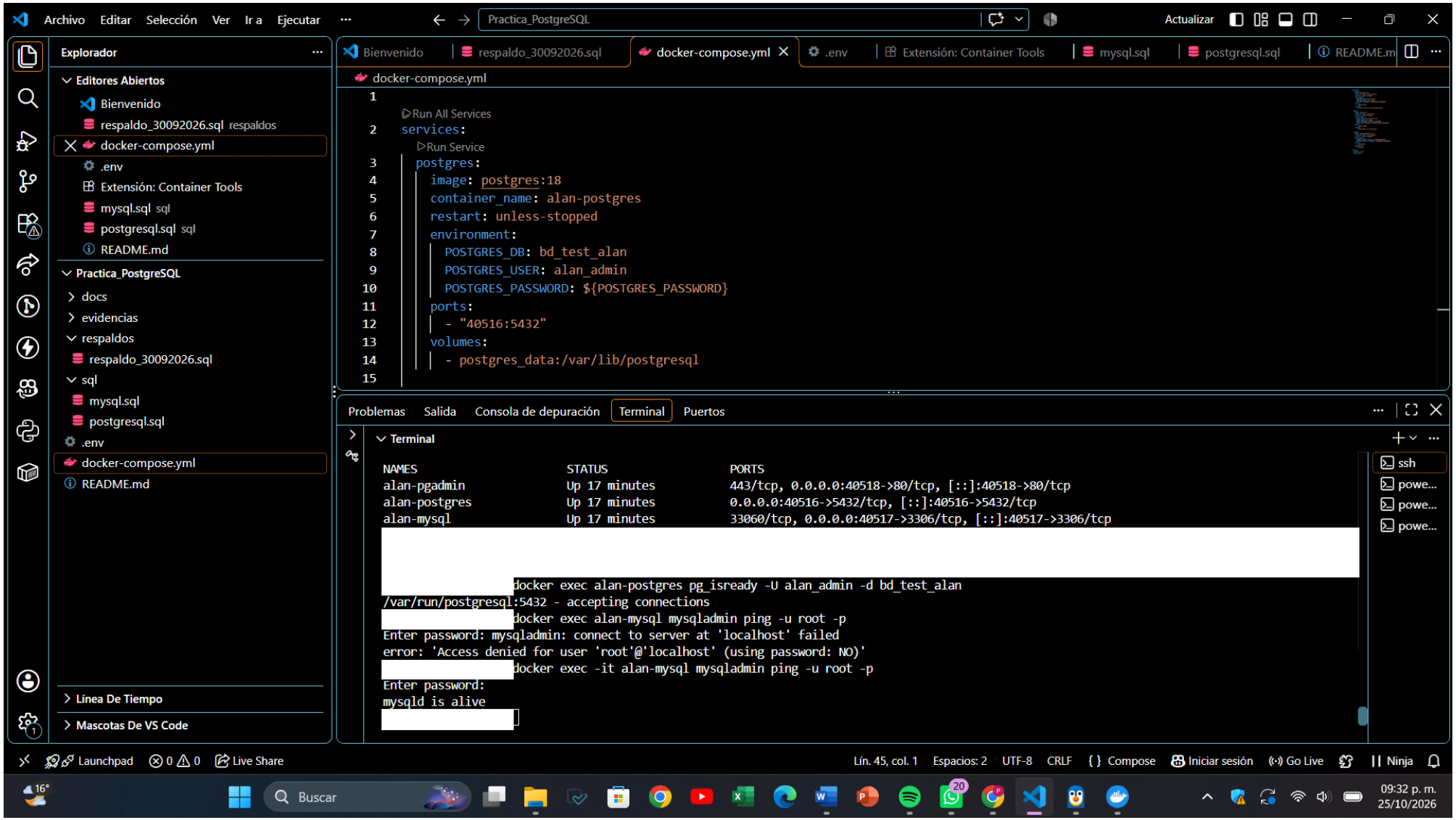
pg_restore --list no restaura ni demuestra recuperación. El dump de base no incluye automáticamente cuentas globales; el administrador entrega el complemento autorizado por canal privado. No ejecutes pg_dumpall indiscriminado en servidor compartido. Los triggers MySQL ausentes no aparecerán en el respaldo. Solo una restauración independiente autorizada permite afirmar recuperación, sin tocar bases activas.

15. Errores y continuación de MySQL

15.1 Autenticación y disponibilidad

mysqladmin ping sin contraseña falló históricamente; el intento interactivo posterior devolvió mysqld is alive. Eso no pedía cambiar contraseñas. PostgreSQL mostró accepting connections. La lectura interna no demuestra cliente externo ni todos los logs.

Figura 32. DOCX-31



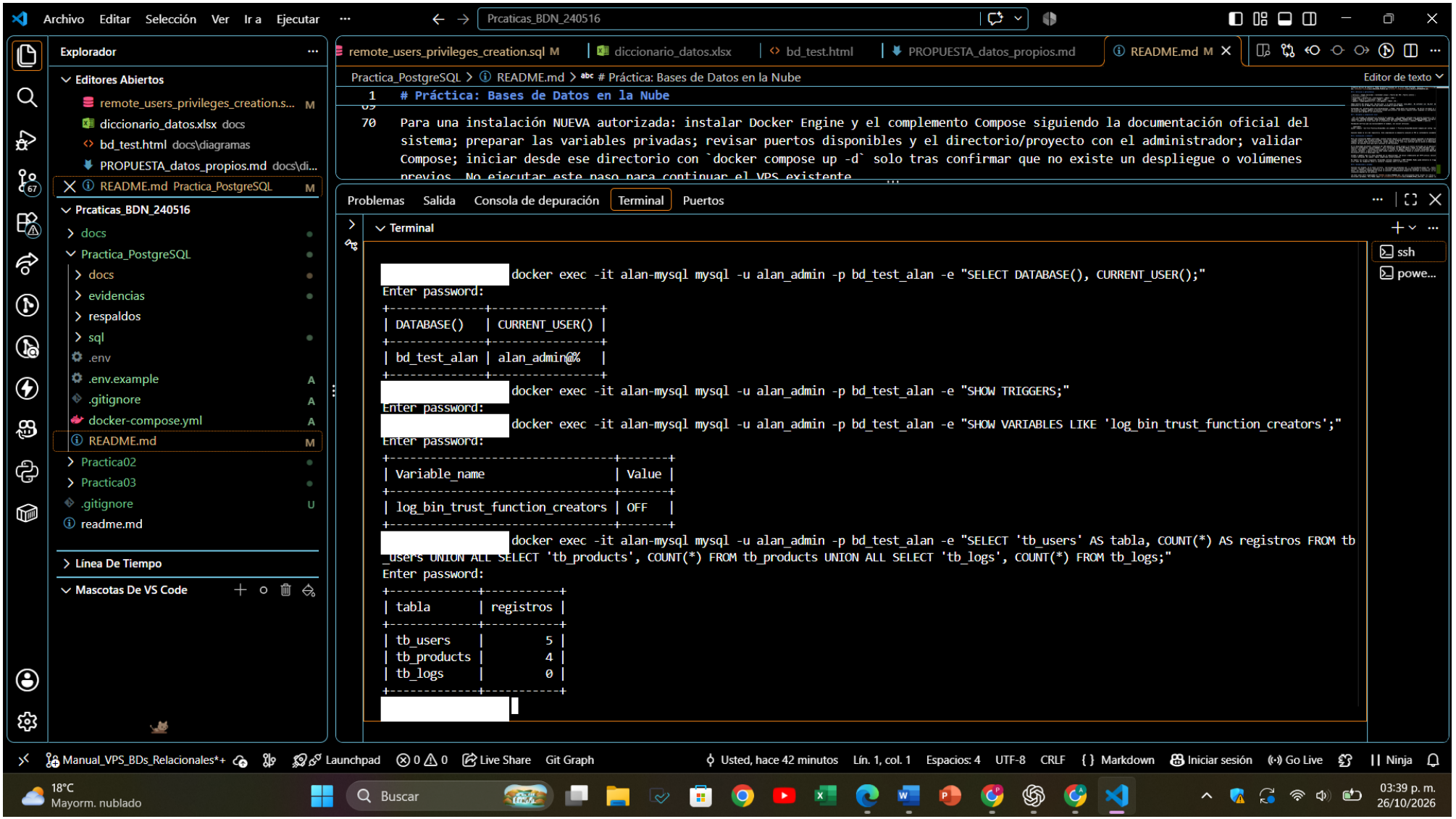
Access denied visible y comprobación interactiva posterior correcta. No se ocultó el fallo ni se repitió aquí.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

15.2 Error 1419

Con registro binario, MySQL puede restringir funciones/triggers por política de confianza y privilegios. El antecedente reportado es 1419; la captura demuestra log_bin_trust_function_creators en OFF. Falta una captura completa del error 1419; no se inventa. El reinicio anterior no confirmó solución. No se cambia configuración global ni se elevan privilegios.

Figura 33. CAPS-13



OFF, consulta sin triggers y conteos MySQL 5/4/0. Demuestra el bloqueo, no su solución.

Evidencia histórica; no se repitió la operación. Copia documental protegida cuando corresponde; original conservado.

15.3 Secuencia que depende del administrador

1. Identificar alan-mysql, bd_test_alan y 40517, sin tocar instancias ajenas.

2. Revisar versión, binlog, confianza y permisos desde sesión autorizada.

Comando / consulta (sql). Revisar el lugar indicado antes de ejecutar.

```
SELECT VERSION(), DATABASE(), CURRENT_USER();

SHOW VARIABLES LIKE 'log_bin';

SHOW VARIABLES LIKE 'log_bin_trust_function_creators';

SHOW GRANTS FOR CURRENT_USER;

SHOW TRIGGERS FROM bd_test_alan;
```

3. Decidir y autorizar el mecanismo permitido de configuración/creación, y comprobarlo en esta instancia. Aquí no se prescribe SET GLOBAL ni privilegios globales.

4. Obtener respaldo autorizado e inspeccionar datos/definiciones. Preparar solo triggers/vistas faltantes, sin reimportar todo mysql.sql ni reemplazar objetos correctos.

5. Tras ejecución autorizada, verificar seis triggers y definiciones de las dos vistas: M02.

6. En operaciones oficiales nuevas autorizadas, comprobar evento correcto por operación y vistas coherentes: M03. No crear cargas nuevas independientes ni duplicar historial.

Estado: bloqueado, sin nueva evidencia de solución. La aclaración docente de permisos y casos es independiente.

16. Git y entrega

16.1 Revisar sin perder avances

Objetivo: conservar archivos e índice. Dónde: PowerShell, raíz del repositorio. Rama: Manual_VPS_BDs_Relacionales. No se cambia ni se crean commits aquí.

Comando / consulta (powershell). Revisar el lugar indicado antes de ejecutar.

```
git branch --show-current
```

```
git status --short
```

```
git diff --stat
```

```
git diff --cached --name-status
```

Distingue cambios de trabajo e índice. No uses git add . ni descarte; no incluyas cambios ajenos de prácticas anteriores. .env está ignorado, no seguido ni preparado. Originales privados y respaldos quedan fuera de publicación.

16.2 Preparar la entrega futura

1. Revisa PDF, evidencias seguras, Compose sin valores resueltos y SQL.

2. Completa portada y conclusiones. Confirma el nombre oficial Reporte_Unidad1_Matricula.pdf; se conserva por ahora el solicitado, sin copias redundantes.

3. Autoriza selección exacta de archivos. Solo después se preparan rutas explícitas y se revisa el diff del índice antes de confirmar.

4. Repositorio privado, commits y fusión son requisitos pendientes. No se realizó commit, push, merge ni reescritura de historial.

Falta I12 para privacidad/commits/fusión autorizada. La rama ya aparece en capturas: no repetir solo por eso.

17. Conclusiones, limitaciones y referencias

17.1 Borrador editable

[PERSONALIZAR] Comprendí que un contenedor activo no garantiza conexión SQL ni permisos. Distinguir Windows, VPS, servicios y clientes ayuda a interpretar puertos y pgAdmin. Debo indicar con mis palabras qué realicé personalmente y qué apoyo recibí.

[CONFIRMAR EXPERIENCIA] PostgreSQL tiene INSERT/UPDATE y trazabilidad históricos. Autenticación y error 1419 permitieron distinguir disponibilidad, privilegios y política del servidor. No puedo afirmar auditoría MySQL funcional hasta verificarla con el administrador.

[COMPLETAR VALORACIÓN] Mi siguiente trabajo es acreditar conexiones externas y permisos, aclarar casos faltantes y obtener respaldos privados comprobados. Añadiré lo aprendido y qué haría diferente; este borrador no sustituye mi reflexión del indicador 52.

17.2 Qué falta para entregar

Recursos/cuentas/permisos VPS, persistencia/logs, firewall y clientes externos, roles/cuentas/privilegios reales, restricciones de clase, auditoría MySQL, diez casos, entrega segura de accesos, portada, conclusiones y Git final. Los históricos PostgreSQL no demuestran un DELETE no observado ni cuotas por actor. CAPTURAS_PENDIENTES.md prioriza lo realmente faltante.

No se cargaron productos, crearon tablas, cambiaron permisos, repitieron UPDATE ni operaron servicios remotos. Esta guía explica el trabajo sin aparentar cumplimiento.

17.3 Referencias y anexos

1. Lista_Verificacion_Unidad_1_Bases_Datos_Nube.pdf: siete páginas, 56 indicadores y entregables.

2. Foto manuscrita: ocho apartados y respaldos; transcripción en CASOS_SIMULACION.md.

3. caps.docx y documento de instalación PostgreSQL: fuentes históricas privadas, sin publicar originales sensibles.

4. Compose, SQL y validadores de objetos/permisos/historial existentes. Tener archivos no acredita ejecución.

5. Docker: https://docs.docker.com/engine/install/debian/ y https://docs.docker.com/reference/cli/docker/compose/up/.

6. pgAdmin: https://www.pgadmin.org/docs/pgadmin4/9.17/connecting.html. Workbench: https://dev.mysql.com/doc/workbench/en/wb-mysql-connections-new.html.

7. MySQL: https://dev.mysql.com/doc/refman/8.4/en/stored-programs-logging.html y https://dev.mysql.com/doc/refman/8.4/en/mysqldump.html.

8. PostgreSQL: https://www.postgresql.org/docs/18/app-pgdump.html y https://www.postgresql.org/docs/18/app-pgrestore.html.

Anexos existentes: CUMPLIMIENTO_RUBRICA.md/.csv contiene los 56 indicadores; CASOS_SIMULACION.md/.csv la matriz por actor/motor; RESPALDOS_PREPARACION.md los bloques completos; evidencias/INVENTARIO_GENERAL.csv las 57 imágenes. Las 31 copias técnicas están en evidencias/seguras/Evidencias_tecnicas_redactadas.pdf; las dos Docker seguras, en evidencias/seleccionadas. Las figuras se insertan junto a sus procedimientos, no solo en anexos.